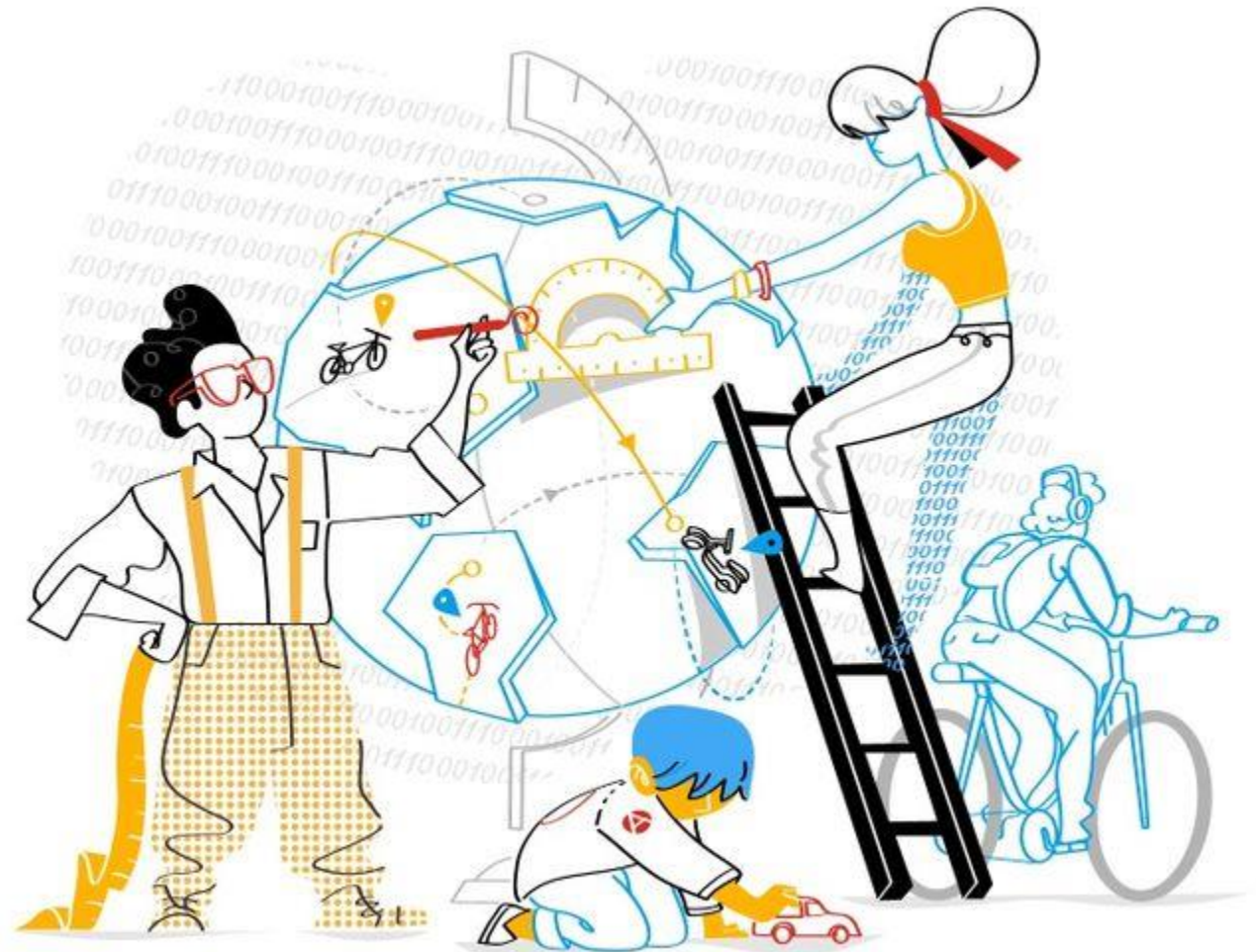


UNIDAD 2.3-Desarrollo y métricas ágiles

Ingeniería de Software

Ing. Mónica Colombo
Ciclo Lectivo 2025

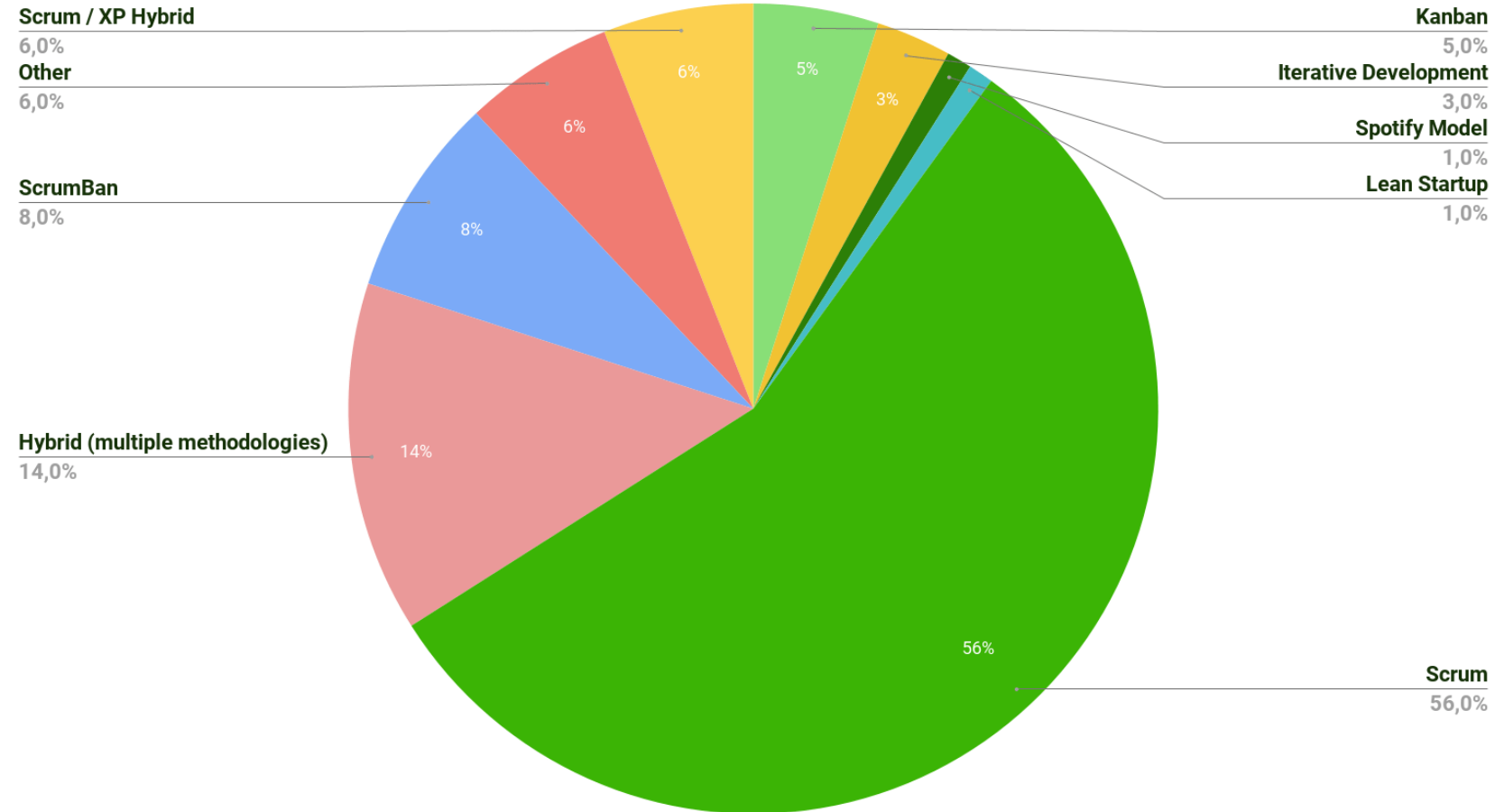


Contenidos: 2.3-Desarrollo y Métricas Agiles

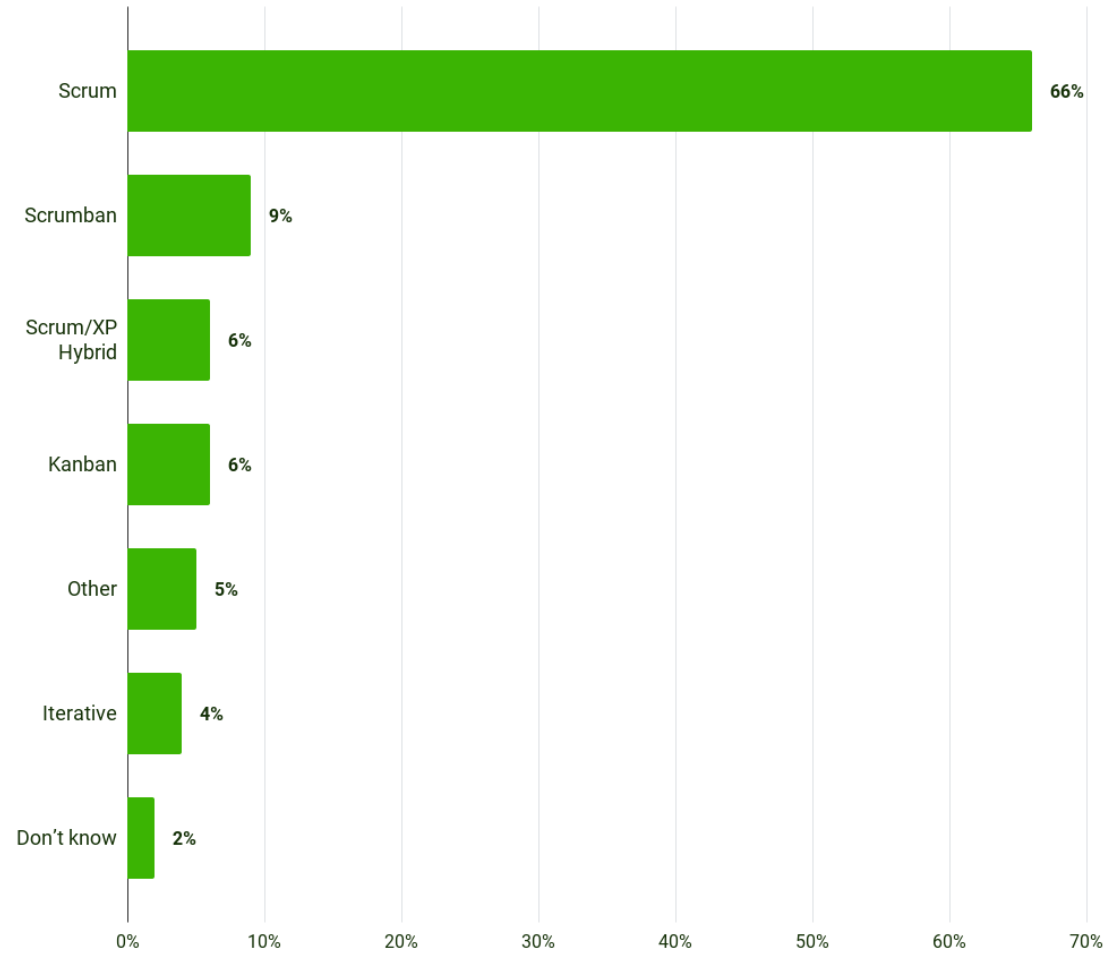
- Métodos.
- Desarrollo Agil.
- Ingeniería de Software agil.
- Trabajo en equipo ágil.
- Realidad de la arquitectura de Software.
- Arquitecto de sistemas y sus funciones en un proyecto.
 - Responsabilidades y tareas.
 - Etapas.
 - Tipos de arquitecto.
 - Ejemplos de arquitectura real y compleja.
- Estimación Agil & Métricas ágiles
- Métricas ágiles para optimizar el delivery.
 - Velocidad.
 - Trabajo.
 - Tiempo.
 - Reportes.



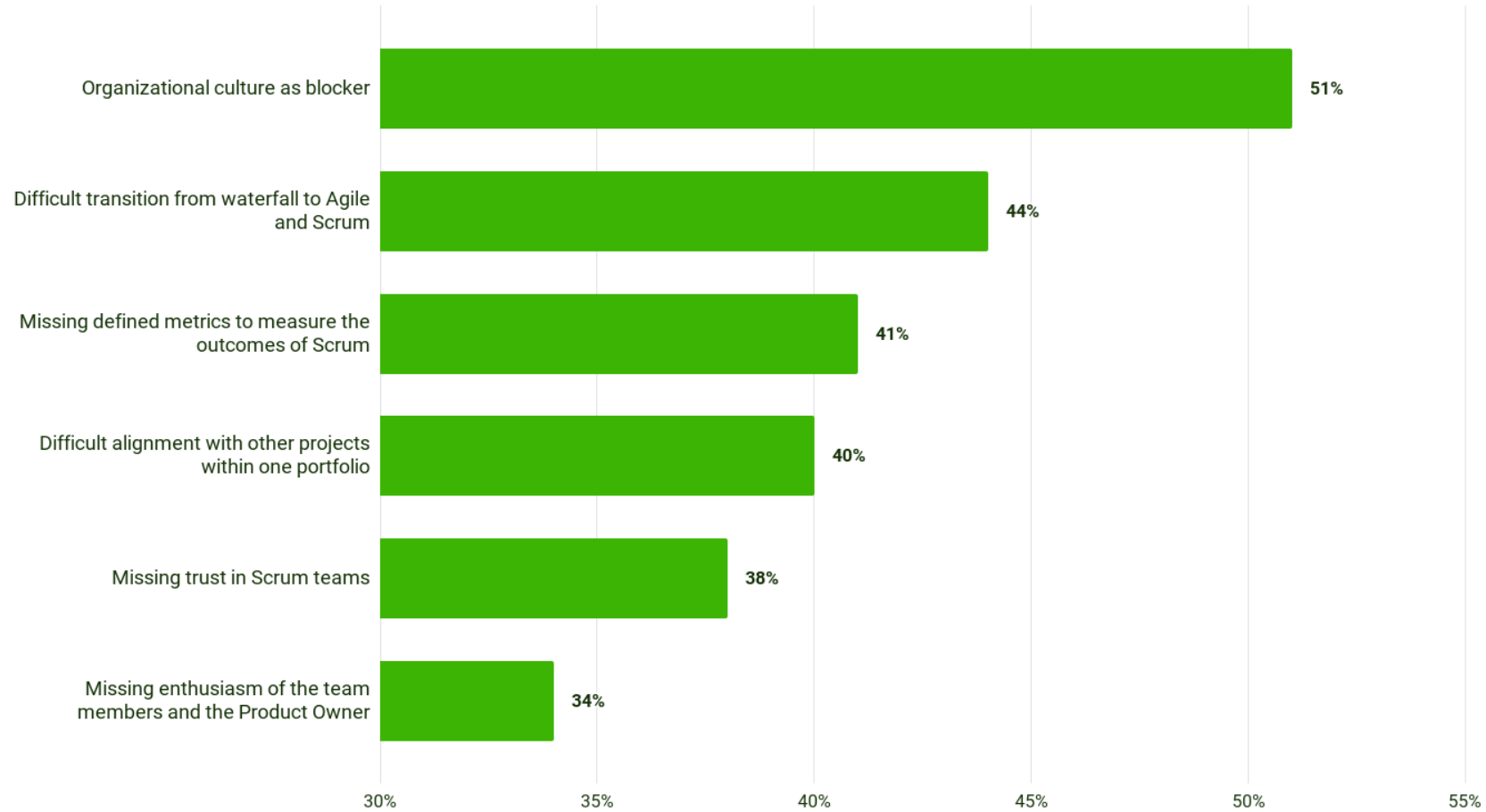
How popular is Scrum?



Which agile methodology do you follow most closely at the team level?



Challenges with the implementation of Scrum



Recomendación: menos métricas es mejor!

- El uso de muchas métricas puede complicar la toma de decisiones y dificultar la identificación de problemas importantes en el proceso.
- Consensuarlas con el clientes, que visibilidad necesita para ver avances y mejoras
- Un conjunto de métricas clave le proporciona al equipo una **visión clara y concisa de las cosas**
- Suelen ser accionables
- **Hay que mantenerlas!** Se deben actualizar y alimentar para poder medirlas con una frecuencia/tiempo

Un ejemplo con cuatro métricas:

- * Frecuencia de despliegue (ej. 120 despliegues)
- * Tiempo de entrega de los cambios (ej. 69,16 horas)
- * Tasa de fallo de los cambios (ej. 9,25 % cada 100%)
- * Tiempo de mejora de la restauración (0,51 horas)



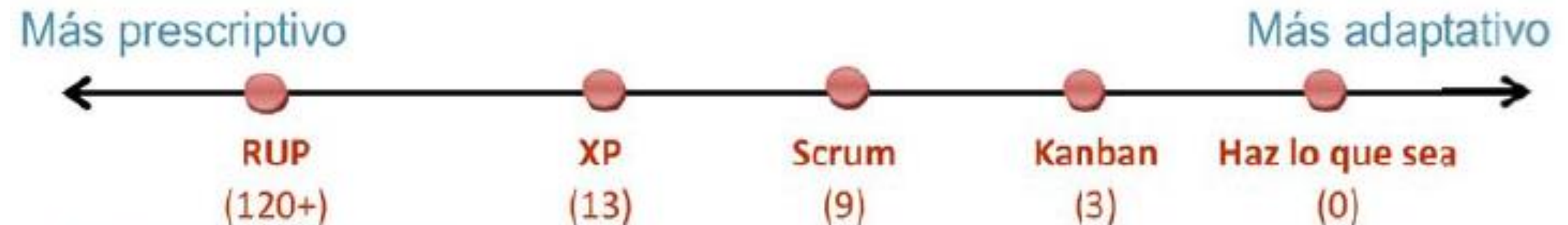
Métodos de Desarrollo Ágiles

- Como cualquier herramienta, **Scrum y Kanban no son ni perfectas ni completas. No te dicen *todo* lo que tienes que hacer, solo proporcionan ciertas restricciones y directrices.**
 - Scrum te obliga a tener iteraciones de duración fija y equipos interdisciplinarios
 - Kanban te obliga a usar tableros visibles y a limitar el tamaño de tus colas.
- Curiosamente, **el valor de una herramienta es que *limita tus opciones*.**

Las métricas se eligen: según el Proyecto y qué se necesita medir/reportar al management del Proyecto y cliente, que nos permita medir la calidad, el cumplimiento de objetivos, hallar errores, y oportunidades de mejora.
y que se puedan realimentar y actualizar.

Métodos de Desarrollo Ágiles

- Los métodos ágiles se denominan a veces los **métodos ligeros**, en concreto, porque son menos restrictivos que los métodos tradicionales. De hecho, el primer principio del Manifiesto Ágil es "Individuos e interacción"
- Scrum y Kanban son muy adaptables, pero en términos relativos Scrum es más restrictivo que Kanban. Scrum te da más limitaciones, y así deja menos opciones abiertas. Por ejemplo Scrum prescribe el uso de iteraciones de duración fija, Kanban no, es sobre procesos y herramientas".



Métodos-Frameworks de Desarrollo Ágiles

SCRUM

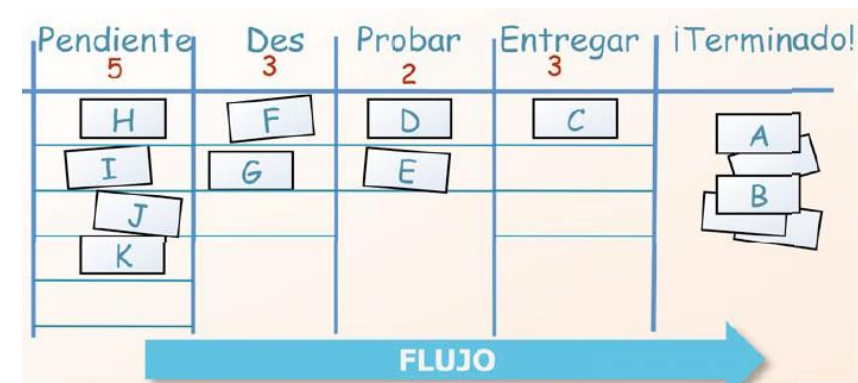
- Desarrollada por Ken Schwaber, Jeff Sutherland y Mike Beedle.
- Define un **marco para la gestión de proyectos**, que se ha utilizado con éxito durante los últimos 20 años.
- Sus principales características se pueden resumir en dos:
 - 1- El desarrollo de software se realiza mediante iteraciones, denominadas sprints, con una duración de 30 días.
 - 2-El resultado de cada sprint es un incremento ejecutable que se muestra al cliente. La segunda característica importante son las reuniones a lo largo proyecto, entre ellas destaca la reunión diaria de 15 minutos del equipo de desarrollo para coordinación e integración.

Métodos-Frameworks de Desarrollo Ágiles

KANBAN

Se basa en:

- 1-Visualiza el flujo de trabajo
 - Divide el trabajo en bloques, escribe cada elemento en una tarjeta y ponlo en el muro.
 - Utiliza columnas con nombre para ilustrar dónde está cada elemento en el flujo de trabajo.
- 2- La limitación del trabajo en curso o Work in Progress, donde se asigna límites concretos a cuántos elementos pueden estar en progreso en cada estado del flujo de trabajo.
- Mide el lead time (tiempo medio para completar un elemento, a veces llamado "tiempo de ciclo"), optimiza el proceso para que el lead time sea tan pequeño y predecible como sea posible.



Métodos-Frameworks de Desarrollo Ágiles



VISUALIZATION

Visualize tasks with digital cards on a kanban board

WIP LIMITS

Discourage multitasking by limiting the number of tasks

WORKFLOW

Define project stages clearly to improve efficiency

Factors

You don't have any processes in place and want to try out agile.

Yes

No

Your projects need to follow strict project deadlines.

Yes

No

You're looking for ways to make your existing project management processes more efficient.

No

Yes

Your primary goal is to find ways to improve employee productivity and output.

No

Yes

Métodos-Frameworks de Desarrollo Ágiles

XP- Programación Extrema

- Pura lógica, es la palabra con que describen la **Programación Extrema** algunos conocedores y desarrolladores, las prácticas de este método están basadas en la simplicidad, la comunicación, la valentía y el reciclado continuo de código.
- **XP** es la integración de las prácticas de métodos tradicionales resumiendo o utilizando lo más práctico y eficaz, sin olvidar la añadidura de características importantes mencionadas en el manifiesto ágil.
- Es un cambio revolucionario **llevar las prácticas al extremo**, teniendo como fin la satisfacción del cliente, Software de calidad, integración del cliente en el equipo de trabajo(cliente in situ), construir con pruebas, adaptación a los cambios ya sean de tecnología o de metáforas dentro del negocio, liberar temprano y trabajar en equipo.

Métodos-Frameworks de Desarrollo Ágiles

Proceso XP (cont.)

- El ciclo de desarrollo consiste (a grandes rasgos) en los siguientes pasos:
 1. El cliente define el valor de negocio a implementar.
 2. El programador estima el esfuerzo necesario para su implementación.
 3. El cliente selecciona qué construir, de acuerdo con sus prioridades y las restricciones de tiempo.
 4. El programador construye ese valor de negocio.
 5. Vuelve al paso 1.
- En todas las iteraciones de este ciclo tanto el cliente como el programador aprenden. No se debe presionar al programador a realizar más trabajo que el estimado en el release, ya que se perderá calidad en el software o no se cumplirán los plazos. De la misma forma el cliente tiene la obligación de manejar el ámbito de entrega del producto, para asegurarse que el sistema tenga el mayor valor de negocio posible con cada iteración.
- El ciclo de vida ideal de XP consiste de seis fases: Exploración, Planificación de la Entrega (Release), Iteraciones, Producción, Mantenimiento y Muerte del Proyecto

Métodos-Frameworks de Desarrollo Ágiles

Crystal Methodologies

- Se trata de un conjunto de metodologías para el desarrollo de software caracterizadas por estar **centradas en las personas que componen el equipo y la reducción al máximo del número de artefactos producidos.**
- Han sido desarrolladas por Alistair Cockburn.
- El desarrollo de software se considera un juego cooperativo de invención y comunicación, limitado por los recursos a utilizar.
- El equipo de desarrollo es un factor clave, por lo que se deben invertir esfuerzos en mejorar sus habilidades y destrezas, así como tener políticas de trabajo en equipo definidas. Estas políticas dependerán del tamaño del equipo, estableciéndose una clasificación por colores, por ejemplo Crystal Clear (3 a 8 miembros) y Crystal Orange (25 a 50 miembros).

Métodos-Frameworks de Desarrollo Ágiles

Feature -Driven Development (FDD)

- Define un proceso iterativo que consta de 5 pasos.
- Las iteraciones son cortas (hasta 2 semanas). **Se centra en las fases de diseño e implementación del sistema partiendo de una lista de características que debe reunir el software.** Sus impulsores son Jeff De Luca y Peter Coad.

Lean Development (LD)

- Definida por Bob Charette's a partir de su experiencia en proyectos con la industria japonesa del automóvil en los años 80 y utilizada en numerosos proyectos de telecomunicaciones en Europa.
- En LD, los cambios se consideran riesgos, pero si se manejan adecuadamente se pueden convertir en oportunidades.
- Busca eliminar el trabajo y el desperdicio innecesarios, mantener a los equipos enfocados y reducir los tiempos de comercialización.

Algunos desafíos.....

- El entendimiento y la promoción de los roles
 - Cubrir las responsabilidades asociadas al rol Product Owner
 - Scrum Master es un rol que exige experiencia y que sea un facilitador
 - El servicio entre roles: SM con PO, con Dev Team y con la organización
 - El rol de tester ágil
 - La disciplina de las ceremonias o eventos
 - Eventos frecuentes de planificación
 - Evento diario del equipo completo
 - Demos de aprobación
 - Retrospectivas para evaluar el Sprint, el desempeño del equipo, y sus formas de trabajo
- 



Ingeniería de Software Ágil

- Hacer ingeniería de software ágil es un enfoque que prioriza la flexibilidad, colaboración y entrega de valor, basado en los principios del Manifiesto ágil.
 - *Es desarrollar software de valor, de modo iterativo e incremental, donde los requisitos y soluciones evolucionan con el tiempo, haciendo entregas frecuentes de calidad y valor, trabajando con equipos auto-organizados y multidisciplinarios que mejoran, inmersos en un proceso compartido de toma de decisiones interactiva y dinámica.*
- Y, lo más importante, es considerar que la ingeniería es, además de una práctica científica, es *una práctica humana* → *construir software se trata de relaciones humanas en un sistema social*. P
 - ★ Por eso, hay que construir relaciones humanas de valor, que habiliten la entrega continua de valor.

Arquitecto de Sistemas

Arquitecto: tiene la responsabilidad de investigar, evaluar y seleccionar las mejores alternativas tecnológicas para atender las necesidades específicas del negocio a un costo razonable.



Entrevista: Robert C. Martin



Entrevista: Martin Fowler



Entrevista a un arquitecto (Tata Consulting)

Entrevista a un arquitecto (Tata Consulting)

Implicaciones de no seguir un proceso conocido de modelado

La arquitectura es una abstracción de un sistema.

Los sistemas pueden tener y tienen una estructura.

Todo sistema tiene una arquitectura.

Si no se desarrolla la arquitectura, se obtendrá una de todas formas, pero puede no ser la adecuada.

La funcionalidad es en gran medida original y referido a los requisitos de calidad:

- Funcionalidad: capacidad del sistema de hacer lo que se pretendía que hiciese
- Los sistemas se descomponen en elementos para lograr variados propósitos, más allá de la funcionalidad:
 - Las opciones de arquitectura promueven ciertas cualidades al tiempo que implementan la funcionalidad deseada.



Consecuencias de las decisiones de Arquitectura de Software

CONSECUENCIAS

La medida en que un sistema alcanza sus requisitos de calidad depende de las decisiones de arquitectura

la arquitectura es crítica para alcanzar los atributos de calidad

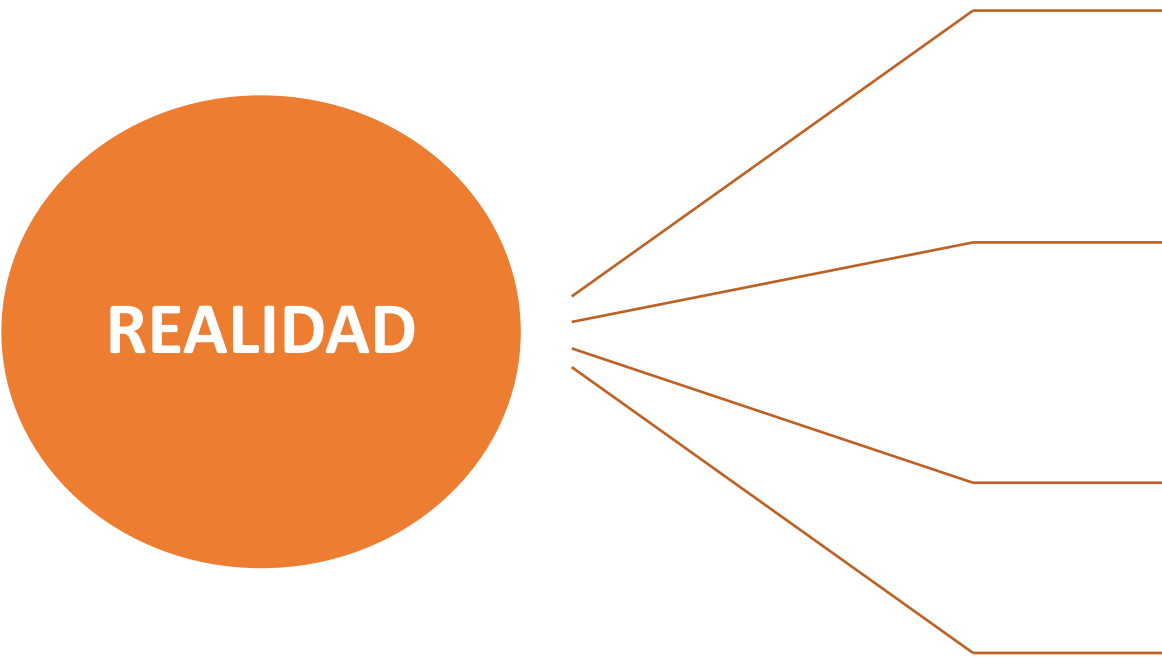
las cualidades y especificaciones de un producto deben diseñarse como parte de la arquitectura

un cambio en la estructura que mejora una cualidad suele afectar las otras cualidades

la arquitectura sólo puede permitir, (no garantizar) que cualquier requisito de calidad se alcance



Realidad sobre Arquitectura de Software



Los requisitos de atributos de calidad son las principales guías para el diseño de la arquitectura.

La medida en que el desarrollo de un sistema alcance sus requisitos de atributos de calidad depende de las decisiones de arquitectura, así como para su posterior mantenimiento.

El desarrollo requiere ser guiado por las decisiones de arquitectura.

La arquitectura pretende evitar que se repitan errores o inconsistencias conocidas.

- Los arquitectos deben identificar e involucrar activamente a los interesados de modo de:
 - ✦ comprender las restricciones reales del sistema
 - ✦ administrar las expectativas de los interesados
 - ✦ negociar las prioridades del sistema
 - ✦ tomar decisiones de compromiso.



Restricciones de Arquitectura de Software

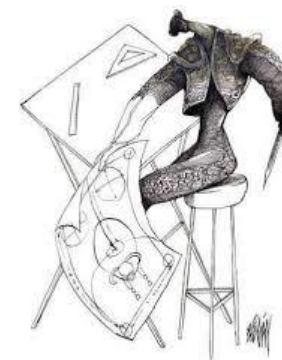
RESTRICCIONES:
(constraints) imponen
condiciones sobre la
arquitectura que
normalmente **no son
negociables.**

Limitan el rango de alternativas
de decisión del arquitecto

Algunas veces hace la vida más
fácil para el arquitecto, en otras
lo complica.

Se pueden clasificar según su
naturaleza: Negocio, Desarrollo,
Tiempo, Costo, etc.

Constraint	Architecture Requirement
Business	The technology must run as a plug-in for MS BizTalk, as we want to sell this to Microsoft.
Development	The system must be written in Java so that we can use existing development staff.
Schedule	The first version of this product must be delivered within six months.
Business	We want to work closely with and get more development funding from <i>MegaHugeTech Corp</i> , so we need to use their technology in our application.



Etapas del proceso de Arquitectura de Software

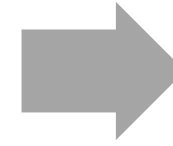
Definir los requerimientos

- Involucra **crear un modelo desde los requerimientos** que guiarán el diseño de la arquitectura basado en los atributos de calidad esperados



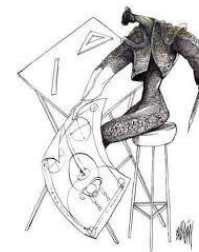
Diseño de la Arquitectura

- Involucra **definir la estructura y las responsabilidades de los componentes** que comprenderán la arquitectura de Software



Validación

- Significa **“probar” la arquitectura**, típicamente pasando a través del diseño contra los requerimientos actuales y cualquier posible requerimiento a futuro.



Influencia de los Interesados

Gerente de la compañía



Bajos costos,
ocupar personal,
aumentar el valor
de los activos
corporativos

Gerente de
Producto



Elementos
atractivos, terminar
rápido, comparable
a la competencia

Usuario
final



Comportamiento,
performance,
seguridad,
confiabilidad,
usabilidad

Ingeniero de Soporte



Modificabilidad

Cliente



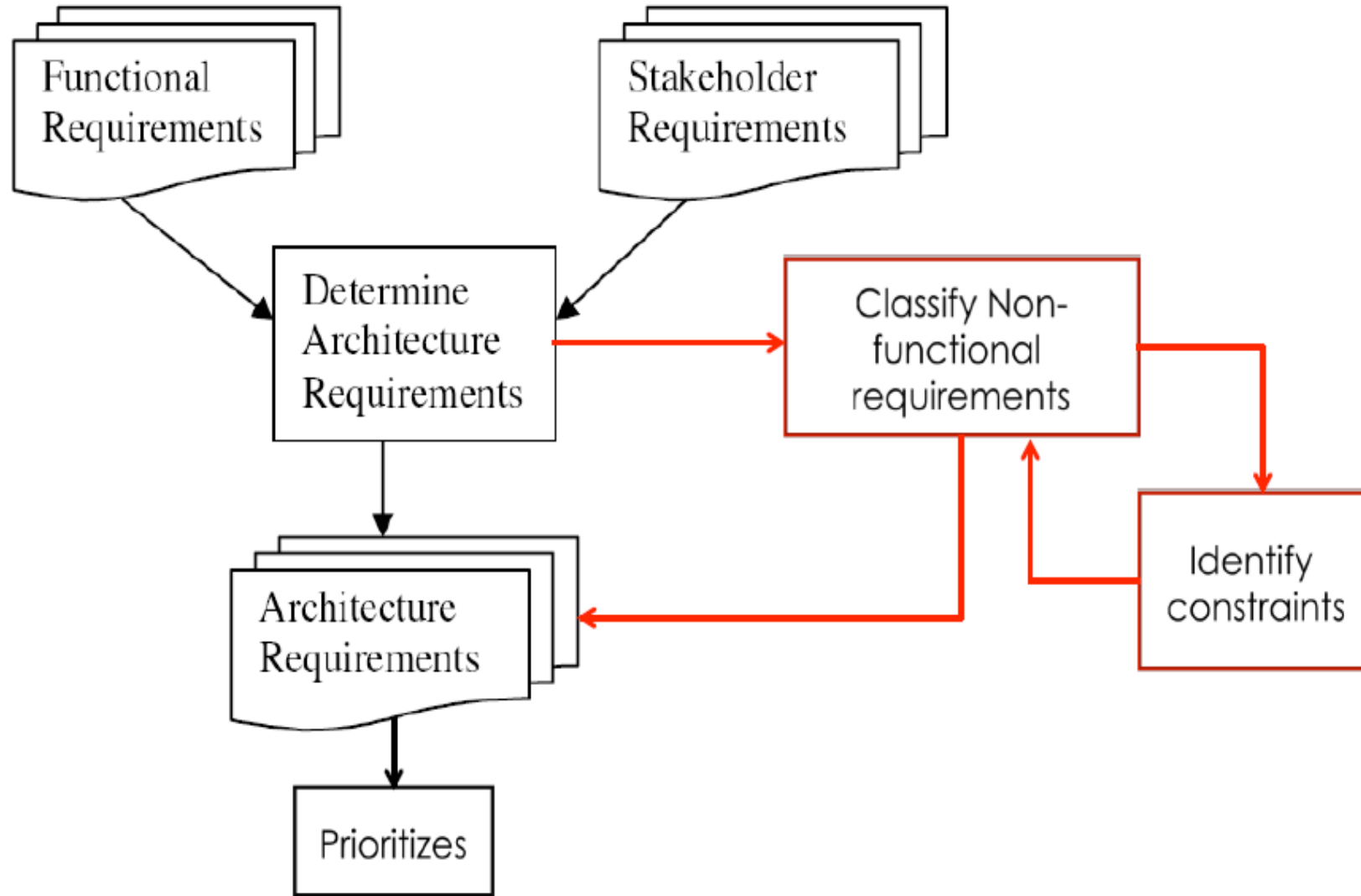
Bajos costos,
terminar rápido,
sin muchos
cambios

Arquitecto



¿Cómo puedo hacer para que
el sistema tenga todo esto?

Identificar requerimientos



Importancia de la arquitectura de software (por aqui voy)

- Una sólida arquitectura de software permite planificar con detalle **el funcionamiento de una aplicación** antes de iniciar con el proceso de desarrollo, establecer tiempos, así como determinar los recursos económicos y humanos que se necesitarán durante todo el proceso.
- En este sentido, es indispensable hacer ciertos cuestionamientos para precisar el funcionamiento final del software. Las más importantes:

COSTO	Cuánto se está dispuesto a invertir en el desarrollo y mantenimiento del sistema? Cuál es el presupuesto? Hay diversos patrones que pueden llegar a ser más complejos, con una infraestructura mayor, lo que puede hacer el proceso de desarrollo más irregular.
DURACIÓN DEL DESARROLLO	Contar con un estimado del tiempo pensado para el desarrollo, además de tomar en cuenta la fecha tentativa de entrega o salida al mercado de la aplicación.
CANTIDAD DE USUARIOS	Es decisivo cuando se está planificando la arquitectura de software: Cuántos usuarios soportará? Es una aplicación web? Es responsive? Es mobile? Es stand-alone? Esto ayuda a orientarse hacia un patrón más o menos distribuido (uno de capas) o en un broker (el más distribuido, ej: Apache Kafka, RabbitMQ). Y esto ayuda a aumentar/reducir el acoplamiento
GRADO DE AISLAMIENTO	Funciona de forma aislada o se integra con otros productos?, o permitirá integraciones de terceros? Es importante elegir una arquitectura que dé la maniobrabilidad necesaria para el producto.

Arquitecto de sistemas

Un arquitecto debe tener una idea de cómo implementar una arquitectura, pero debe estar "preparado para aceptar cualquier otra forma que cumpla con los objetivos también".



Frederick P. Brooks, Jr

- La definición de la arquitectura **se trata de la introducción de la estructura, directrices, principios y liderazgo de los aspectos técnicos de un proyecto de software.** Son las partes de un sistema que determina la calidad de un sistema: disponibilidad, modificabilidad, performance, escalabilidad, seguridad, testeabilidad y usabilidad.
- Se requiere de alguien que asuma la propiedad del proceso de definición de la arquitectura y esto es sin duda, parte de las competencias del **Arquitecto de Software.**
- **El arquitecto impone limitaciones en el diseño para asegurar que se va a alinear con la estrategia de la empresa y su tecnología. Esto incluye consideraciones como cumplimiento, estándares tecnológicos, eficiencia operativa, calidad, buscando la optimización global del software.**

El arquitecto: dónde/con qué trabaja?



Arquitecto de Sistemas

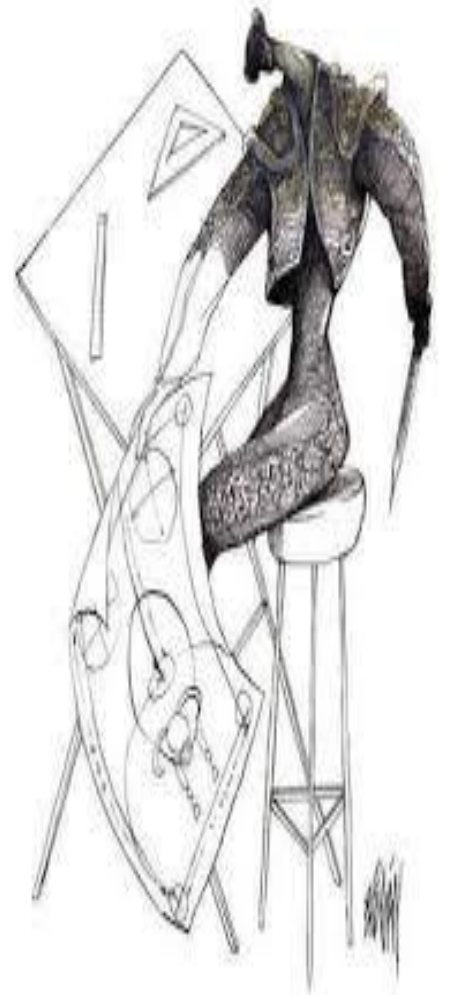
- **La comparación entre la arquitectura tradicional y la arquitectura de sistemas** tiene su fundamento dado que en ambos casos los observadores (usuarios) están pendientes de la arquitectura de la edificación y se preocupan de que se cumplan las diferentes dimensiones (tamaño, mantenibilidad, solidez) a través del ciclo de vida del desarrollo del proyecto

Debe poseer competencias en diferentes dominios: tecnología, diferentes lenguajes de programación, herramientas, gestión en la nube, gestión de requisitos no funcionales, mejora continua, estrategia de negocios, política organizacional, consultoría, facilitador, aseguramiento de la calidad, coach y liderazgo



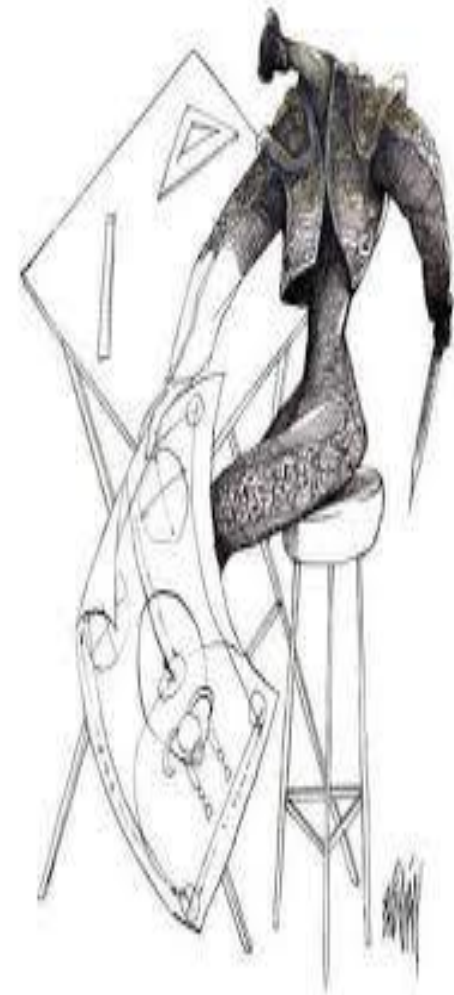
Arquitecto de Sistemas: responsabilidades

- Las responsabilidades del arquitecto de sistemas podrían sintetizarse en:
 - articular la visión arquitectónica
 - conceptualizar y experimentar con diferentes alternativas tecnológicas
 - crear modelos, componentes y documentos de especificación de interfaces
 - validar la arquitectura contra los requerimientos y presunciones del impacto de la alternativa seleccionada sobre la estrategia tecnológica de la organización.



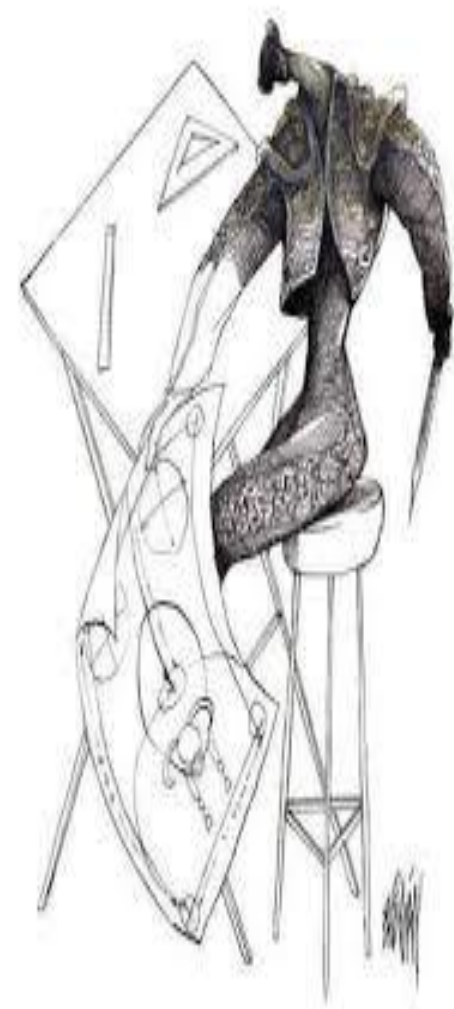
Arquitecto de Sistemas: funciones

- El Arquitecto de Software debe ser una persona con amplios conocimientos técnicos, gran experiencia en programación, liderazgo y que ejerza las siguientes funciones:
 - Gestión de los requisitos no funcionales y definición de la Arquitectura de Software
 - Los requerimientos no funcionales tienen que ser específicos, medibles, alcanzables y comprobables, para poder satisfacerlos (no basta con algo subjetivo como: “el sistema debe ser rápido”), y además hay que saber priorizarlos de manera que todos sean tomados en cuenta.
 - Ej: rendimiento, la escalabilidad, la disponibilidad, auditoría, etc
 - Selección de la Tecnología y por lo tanto, es responsable del riesgo técnico.
 - Mejora continua de la Arquitectura
 - El Arquitecto de Software debe encargarse de la mejora continua de la Arquitectura y a su vez estar abierto a modificarla utilizando las sugerencias o feedback que se pueda obtener de otros miembros del equipo.



Arquitecto de Sistemas: funciones (cont.)

- **Facilitador, líder y formador en aspectos de arquitectura**
 - La arquitectura del software debe ser conocida y entendida no solo por el equipo de desarrollo sino también por otras áreas como seguridad informática, base de datos, operaciones, el equipo de mantenimiento, etc.
 - El Arquitecto de Software **debe asumir la dirección técnica**, para asegurar que todos los aspectos de la arquitectura se estén implementando de manera correcta.
 - Arquitecto de Software **debe proporcionar orientación técnica y dar apoyo al equipo de desarrollo; debe estar preparado para entrenar al equipo en las tecnologías seleccionadas (Formador) y también debe estar abierto a sugerencias.**
- **Aseguramiento de la Calidad**
 - **Garantizar la calidad es parte fundamental del rol**, el cual debe apoyarse en procesos de integración continua que utilicen herramientas automatizadas de análisis de código fuente, pruebas unitarias y cobertura de código, para asegurar el cumplimiento de las normas, políticas y mejores prácticas establecidas.



Arquitecto de Sistemas: en distintas etapas

1-Inicio del proyecto

- Tiene la **responsabilidad de realizar una traducción de las necesidades que expresa un cliente hacia una solución técnica preliminar**, pieza clave para producir una estimación del desarrollo.

2-Estimación del sistema

Debe hacer uso de **habilidades técnicas**

identificar estilos arquitectónicos y tecnologías que sean apropiados para resolver el problema y proponer una solución preliminar

y **no-técnicas**

realizar un análisis de las necesidades del cliente, especialmente desde una perspectiva de negocio y poder explicar la solución técnica que propone a los stakeholders).

3-Requerimientos del Sistema

- Se involucra con los **requerimientos que influyen en la arquitectura** ("drivers") y particularmente con respecto a los **atributos de calidad del sistema**.
- Identifica atributos de calidad pertinentes para el sistema (alineados a los objetivos de negocio) y que las métricas asociadas estén justificadas

4-Diseño del sistema

- Diseñar la arquitectura busca **establecer una solución técnica que satisfaga, los requerimientos que influyen en la arquitectura**.
- Debe ser **capaz de comunicar el diseño, y su justificación**, documentando al equipo de desarrollo.
- Si se evalúa el diseño de la arquitectura, el arquitecto debe ser capaz de presentar el contexto del problema y el diseño de la arquitectura a un comité de evaluación o auditoría.

5-Desarrollo del Sistema

- Desarrolla las funcionalidades más complejas, *aplica programación segura*, da soporte al equipo e implementa buenas prácticas.
- Completa las partes faltantes del diseño de la arquitectura y corrige según decisiones que hayan resultado ser erróneas.
- Es mentor y explica el diseño del sistema al equipo de trabajo.
- Realiza actividades de aseguramiento de calidad tales como inspecciones de productos de trabajo y code review.
- Al momento de realizar pruebas del sistema, realizar pruebas relativas a los atributos de calidad del sistema.

6-Liberar o implementar el sistema

- Realiza ajustes finos de la aplicación con el fin de lograr un funcionamiento óptimo en el ambiente de Producción.

Arquitecto de Sistemas: categorías

- Dependiendo del tamaño del sistema, es posible que no haya un solo arquitecto que participe a lo largo de todo el proyecto y que, más bien, existan distintos arquitectos especializados que intervienen a distintos momentos del desarrollo o con diferentes objetivos.

Arquitecto de soluciones

- Participa en la concepción, preventa, gestión, desarrollo y operaciones
- Su objetivo es proporcionar soluciones específicas e integrales para una serie de aplicaciones, un proveedor o un programa específico dentro de una organización.

Arquitecto de Sistemas

- Diseña la arquitectura global de sistemas informáticos.
- Toma decisiones de diseño que van más allá del software e involucran hardware, siempre orientado a las necesidades del cliente y su negocio
- Evalúan y seleccionan las tecnologías, herramientas y frameworks, eligiendo lenguajes de programación, bases de datos, patrones de diseño y componentes específicos.

Arquitecto Enterprise

- Se especializa en el diseño de una arquitectura empresarial, abarcando sistemas transversales dentro de una empresa y según el tipo de negocio.

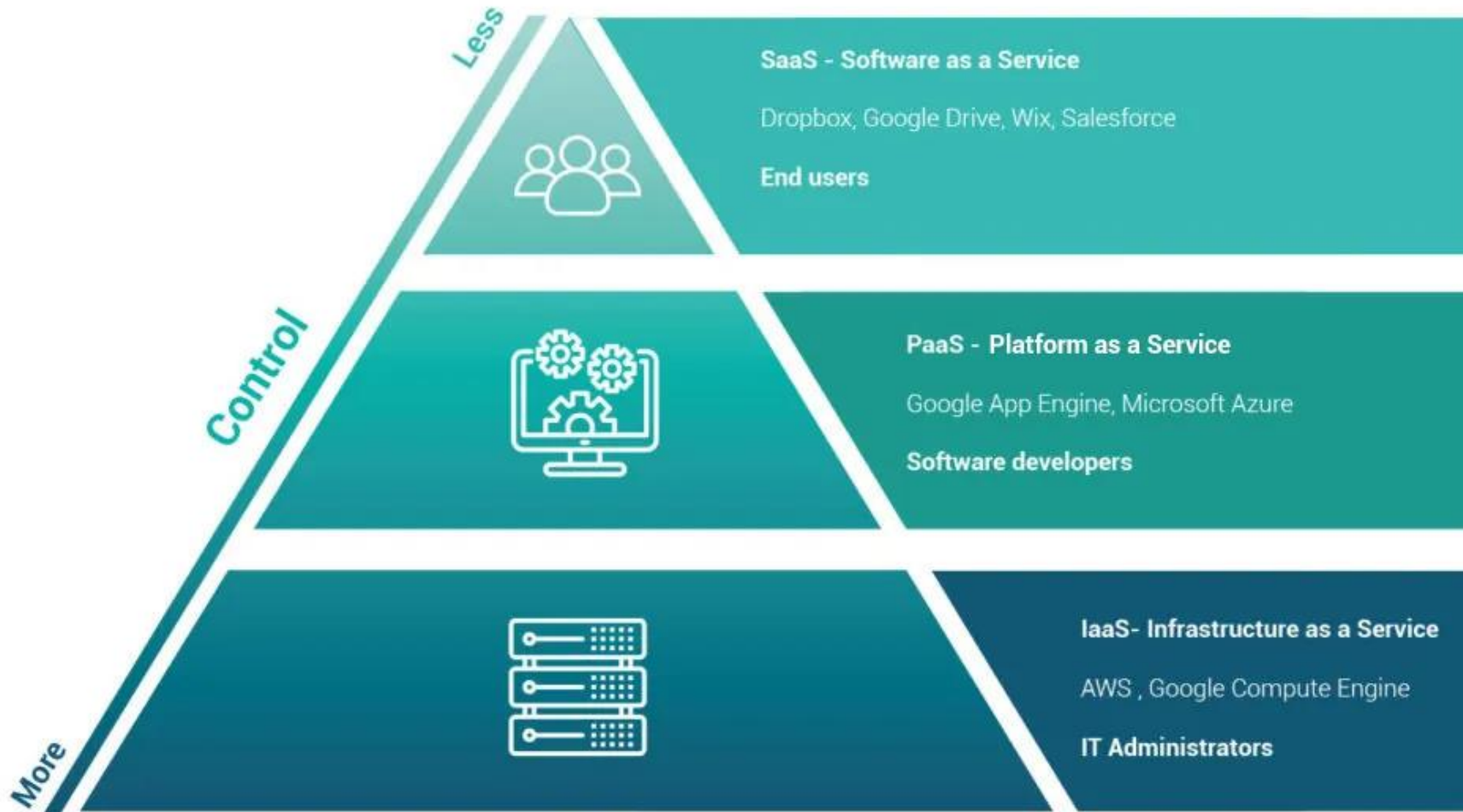
Arquitecto SOA (orientado a servicios), Cloud, IA, Centrado en datos, etc.

- Hay diversas especializaciones a nivel tecnológico. Por ejemplo: el orientado a Cloud diseña arquitecturas para soluciones en la nube, así como su planificación, seguridad y rendimiento.

Arquitectos DevOps

- Analizar, diseñar e implementar estrategias para despliegues continuos para garantizar una alta disponibilidad en los sistemas de producción y preproducción
- Diseña arquitecturas que permitan la integración continua (CI) y la entrega continua (CD) para automatizar la construcción, prueba e implementación del código en entornos similares a los de producción

Arquitecto Cloud Computing (ejemplo: SaaS, Paas y IaaS)



Veamos ejemplos de avisos pidiendo un arquitecto

Como Arquitecto vas a poder definir la arquitectura de los sistemas, tomando decisiones de diseño de alto nivel y estableciendo los estándares técnicos. Vas a tener el desafío de crear la arquitectura con las últimas tecnologías del mercado.

- **En esta posición vas a poder:**

Definir la arquitectura de las plataformas de la compañía

Promover la actualización tecnológica en los proyectos

Determinar los requisitos funcionales y no funcionales teniendo en cuenta las necesidades del negocio

Formar parte de un equipo horizontal que cooperan entre si constantemente

- **Si sos una persona:**

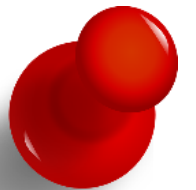
Con excelentes habilidades de comunicación y escucha activa

Pragmática enfocada en los objetivos

Que fomenta el trabajo en equipo y le gusta transmitir conocimientos

- **Si tu perfil incluye:**

- Conocimientos avanzados en Java, NodeJs, JavaScript, HTML
- Conocimientos en Frameworks y librerías como Spring, Angular, ReactJs, TypeScript o VueJs
- Entendimiento en infraestructura en sistemas de queueing: RabbitMQ / Kafka / ActiveMQ / Otros JMS providers
- Tecnologías de orquestación: Mesos, DC/OS, Kubernetes, Docker Swarm
- API gateway / manager: Kong / Apigee / Otros
- IaaS: AWS / Azure / GCE
- Bases de datos relacionales y NoSQL
- Build tools: Maven, SBT, Gradle, Grunt, Gulp



Veamos avisos pidiendo un arquitecto

- Responsable de planear y coordinar los esfuerzos de arquitectura en tiempo para atender a los Squads/equipos de trabajo.
- Proveer experiencia y direccionamiento a los squads durante todo el ciclo de modernización de las aplicaciones, actuar en conjunto con el core team para evaluar las soluciones técnicas de los equipos para garantizar que estas soluciones estén alineadas con la adopción de estándares y tecnologías, responsable de definir la arquitectura de referencia y las estrategias de adopción para entornos de multi-cloud, responsable de ser el nexo con el equipo de arquitectura de un banco en el exterior, desarrollo y negocio.
- Responsable de asegurar que los lineamientos y guías de arquitectura definidos para la plataforma web y/o móvil se materialicen en piezas de software desarrolladas y se integren con el ecosistema de la empresa.
- Tiene autoridad máxima en el diseño de la solución del canal que representa y es el encargado de velar por la integridad end to end del canal que representa.
- +5 años en el rol de arquitecto, con experiencia con clientes en el exterior y disponibilidad para viajar por períodos prolongados.

Experto en:

- Soluciones empresariales end-to-end.
- Gobierno y ciclo de vida de las aplicaciones , metodologías de trabajo, relevamiento de información y oportunidades de mejora.
- Mejores practicas de ingeniería de software
- Industria Bancaria (+5 años)
- Contenedores (Docker) (+3 años)
- Microservicios (+3 años)
- Spring Boot (+3 años)
- Java (+5 años)



Familiarizado con:

- AWS - Arquitecturas de micro front ends - Seguridad a nivel de aplicaciones y soluciones - DevOps - IaaS | PaaS - Metodología agile Scrum y Kanban - Jira

Como Arquitecto vas a poder definir la arquitectura de los sistemas, tomando decisiones de diseño de alto nivel y estableciendo los estándares técnicos. Vas a tener el desafío de crear arquitecturas con las últimas tecnologías del mercado, diseñando soluciones complejas.

• En esta posición vas a poder:

Definir la arquitectura de las plataformas de la compañía
Promover la actualización tecnológica en los proyectos
Determinar los requisitos funcionales y no funcionales teniendo en cuenta las necesidades del negocio
Formar parte de un equipo horizontal que cooperan entre si constantemente
Entender los requerimientos no funcionales. Y definir una arquitectura que cumpla con ellos
Soporte a diversas áreas, como el área comercial estimando oportunidades o siendo representante técnico en reuniones ejecutivas.



• Si sos una persona:

Con excelentes habilidades de comunicación y escucha activa en castellano y en inglés, con un nivel de confort alto hablando con ejecutivos, gerentes, y desarrolladores.
Pragmática enfocada en los objetivos, así como también resiliente y enfocado en la solución para los clientes
Que fomenta el trabajo en equipo y le gusta transmitir conocimientos, con experiencia en auditorios/congresos

• Si tu perfil incluye:

- Conocimientos avanzados en Java, NodeJs, JavaScript, HTML
- Conocimientos en Frameworks y librerías como Spring, Angular, ReactJs, TypeScript o VueJs, Apache cordova, Data APIs (e.g Restful API), DialogScript, Google Cloud/AWS, Microservicios, entre otras.
- Entendimiento en infraestructura en sistemas de queueing: RabbitMQ / Kafka / ActiveMQ / Otros JMS providers
- Tecnologías de orquestación: Mesos, DC/OS, Kubernetes, Docker Swarm
- API gateway / manager: Kong / Apigee / Otros
- IaaS: AWS / Azure / GCE - Bases de datos relacionales y NoSQL
- Build tools: Maven, SBT, Gradle, Grunt, Gulp

A promotional banner for Netflix featuring a collage of various TV show posters. The posters include 'The Wave', 'The Walking Dead', 'Narcos', '13 Reasons Why', 'Prison Break', 'Vikings', 'Iron Fist', 'Bad Moms', '10.0 Earthquake', 'Soy Pasa', 'The Marginal', 'Fifty Shades', and 'SM'. The Netflix logo is in the top left corner.

NETFLIX

See what's next.

WATCH ANYWHERE. CANCEL ANYTIME.

JOIN FREE FOR A MONTH

**Ejemplo de una
arquitectura**



NETFLIX

Dispositivos de streaming

STREAMING

A method of transmitting or receiving data (video and audio) over a computer network as a steady, continuous flow, allowing playback to proceed while subsequent data is being received.

- Ve desde donde quieras, cuando quieras, en miles de dispositivos.
- El software de streaming de Netflix te permite ver al instante contenido de Netflix a través de cualquier [dispositivo con conexión a Internet](#) que cuente con la aplicación de Netflix, incluidos Smart TV, consolas de juegos, reproductores multimedia, smartphones o tabletas.
- También puedes ver Netflix directamente desde tu computadora o laptop.
 - Te recomendamos revisar los [Requisitos del sistema](#) para obtener información sobre los navegadores compatibles.
- La aplicación de Netflix puede venir preinstalada en ciertos dispositivos o, en otros casos, es necesario que los usuarios la descarguen a sus dispositivos.



NETFLIX

Microservicios: Netflix divide su aplicación en cientos de microservicios independientes, cada uno responsable de una función específica, como la autenticación, las recomendaciones de contenido, la reproducción y la facturación.

Frontend: React para las aplicaciones web, Swift para iOS y Kotlin para Android en las aplicaciones móviles.

Backend: Java, SpringBoot y GraphQL para las APIs, y un framework interno llamado DGS para conectarlos.

Bases de datos: EV Cache para la caché de datos en memoria y CockroachDB para una base de datos distribuida y escalable.

Infraestructura: Amazon Web Services (AWS) para la infraestructura en la nube.

Mensajería y streaming: Apache Kafka para colas de mensajes y Apache Flink para streaming de datos en tiempo real.

CDN: red de distribución de contenido (CDN) propia llamada Open Connect para la distribución de contenido de vídeo.

Procesamiento de datos: Apache Spark para procesar y analizar datos en tiempo real y Tableau para visualizar los resultados.

CI/CD: JIRA, Confluence y Spinnaker, una herramienta propia de código abierto para el despliegue continuo.

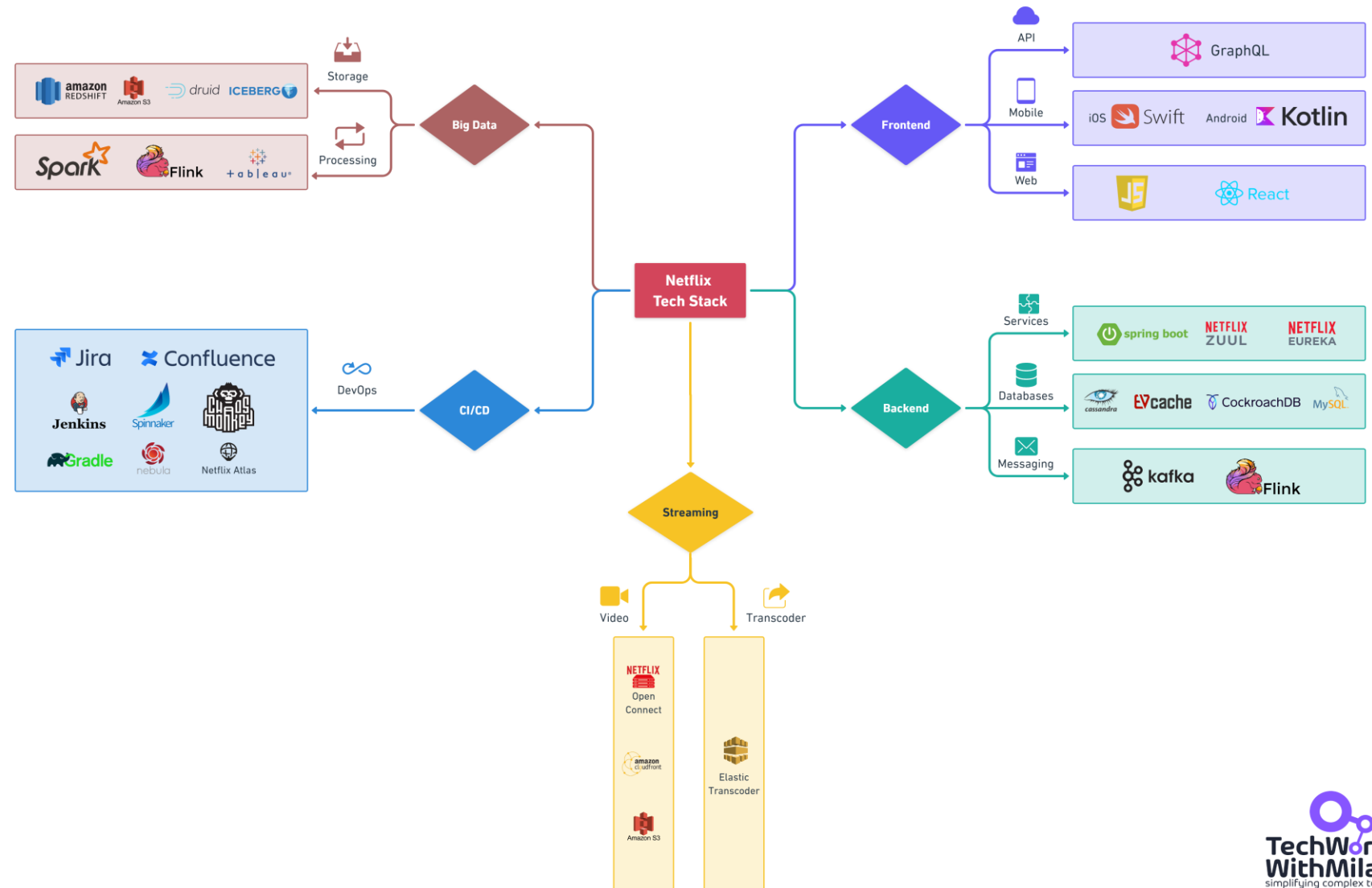
Gateway API: Zuul, un servicio de gateway de enrutamiento dinámico, seguridad y monitorización, es utilizado para las APIs.

Descubrimiento de servicios: Eureka es utilizado para descubrir servicios, equilibrar la carga y manejar fallos.



NETFLIX

NETFLIX Tech Stack



Métricas en equipos

- Sabemos que es muy subjetivo medir a un equipo, sin embargo, **se puede ser subjetivo y tener brújulas o indicadores** que guíen el camino de un equipo ha alcanzar una mejora continúa que se mantenga con el tiempo.
- **Plantear el para qué medimos y qué valor agregan**
- Las métricas sirven para apalancar la **mejora continua**, ayudando a las organizaciones a enfocar a las personas en lo mas importante y en generar valor.
- Para ello es importante **definir mediciones periódicas** para tomar datos sobre el comportamiento del equipo durante el determinado corte.



Estimación Ágil & Métricas ágiles

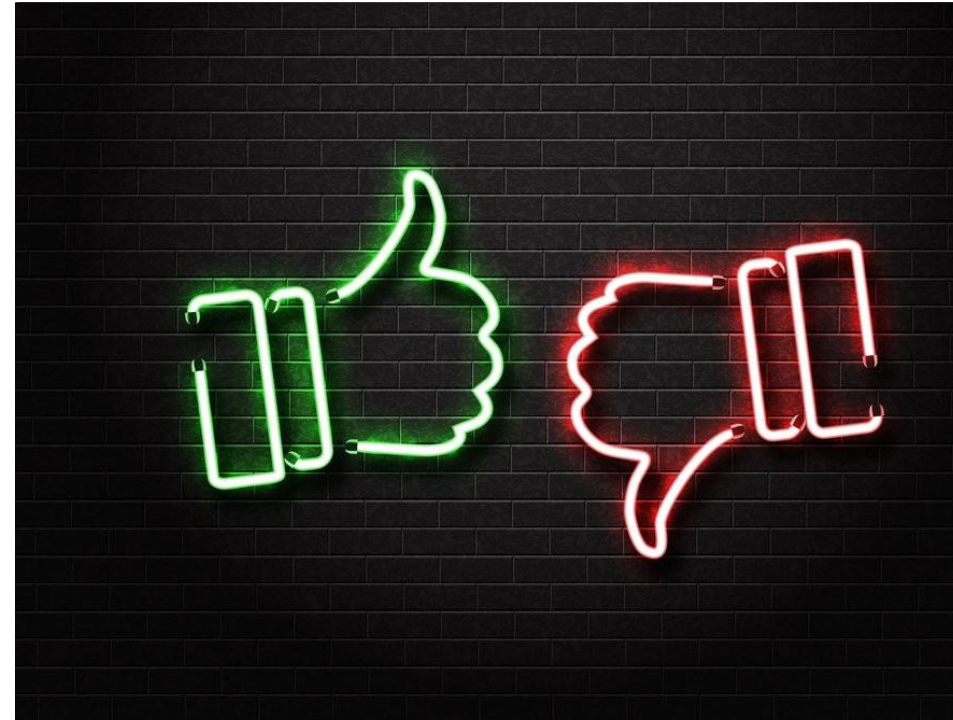
Cómo se calcula el avance de un “proyecto” ágil?

- La pregunta suele venir cuando alguien que sigue trabajando en gestión clásica, se acerca a la gestión ágil y se le caen dos piezas fundamentales de la misma: **la gestión de avance por horas y la fecha fin de proyecto.**
- La gestión de avance por horas se basaba en tener una fecha fin de entrega de algo, y contar de alguna manera las horas, o días, que hay desde el comienzo hasta esa fecha fin y luego, según la fecha en la que estamos, hacer un porcentaje de horas para “saber” como vamos.



Estimación Ágil & Métricas ágiles

- Las métricas son un tema complicado.
 - Por un lado, **todos hemos estado en un proyecto en el que no se ha supervisado ningún tipo de dato, y era complicado decir si íbamos cumpliendo los objetivos para la publicación o si éramos más eficientes según avanzaba el proyecto.**
 - Por otro lado, muchos hemos tenido la mala suerte de participar en proyectos en los que las estadísticas **se usaban como un arma**, enfrentando a un equipo frente al otro o justificando el trabajo obligatorio del fin de semana. Así que no es ninguna sorpresa que **la mayoría de los equipos tengan una relación de amor-odio con las métricas.**
- Sin embargo, no tiene que ser de este modo. Supervisar y compartir métricas ágiles sólidas puede reducir la confusión y arrojar luz sobre el progreso (y retrasos) del equipo a lo largo del ciclo de desarrollo.



Métricas Agiles & gráficas

**1-Sprint
burndown**

3-Velocidad

**5-Diagrama
de flujo
acumulado**

**2-Evolución
de Epicas y
publicaciones**

**4-Gráfico de
control**

Métricas Ágiles

Conoce tu trabajo

- El estado "Done" (Finalizado) solo cuenta la mitad de la historia. **Se trata de crear el producto correcto en el momento oportuno para el mercado adecuado.** Para no desviarse del camino a lo largo del programa, es necesario recopilar y analizar algunos datos.
- En todos los programas ágiles, es importante realizar un seguimiento de las métricas empresariales y ágiles. Las métricas empresariales informan de la adecuación de la solución al mercado, **mientras que las métricas ágiles miden aspectos del proceso de desarrollo.**
- Las métricas empresariales de un programa deben estar fundamentadas en su hoja de ruta.

Para cada iniciativa de la hoja de ruta, incluye varios indicadores clave del rendimiento (KPI) asociados a los objetivos del programa. Además, incluye criterios del éxito para cada [requisito](#) del producto, como la tasa de adopción por los usuarios finales o el porcentaje de código cubierto por pruebas automatizadas. Estos criterios de éxito alimentan las métricas ágiles del programa. **Cuanto más aprendan los equipos, mejor se adaptarán y evolucionarán.**



Consideraciones cuando medimos en Agil

- **Métricas del producto**

- Satisfacción del usuario
- Tasa de rebote
- Costo de adquisición de clients
- Tasa de conversion de la funcionalidad
- Costo de adquisición de clients
- Tiempo del usuario para obtener valor
- Tasa de conversion de usuarios por 1era.vez
- Esfuerzo del usuario
- Tasa de éxito de la funcionalidad
- Rentabilidad del usuario
- Etc.

- **Métricas de proceso** (Scrum, Kanban, etc.)

- **Puntos de Historia**

- **Medir Motivación y Cultura/Ambiente**

- Cómo ser predecible sin estimar
- Herramientas y como implantar

Métricas Ágiles

- Mindset
- Ley de Goodhart

1- Valor (outcome o resultado)

- cualitativas/cuantitativas

2- Eficiencia (output o salida) y Desperdicio

3- Estimación

- Velocidad

4-Equipo y Cultura

5-Cambio

6-Deuda Técnica

- calidad del producto

[HTTPS://WWW.YOUTUBE.COM/WATCH?V=80THWGJ-NHO&T=22S&AB_CHANNEL=JAVIERGARZ%C3%A1S](https://www.youtube.com/watch?v=80THWGJ-NHO&t=22s&ab_channel=JavierGarz%C3%A1s)

Métricas Ágiles: Cómo usar métricas ágiles para optimizar la entrega

1- Sprint burndown

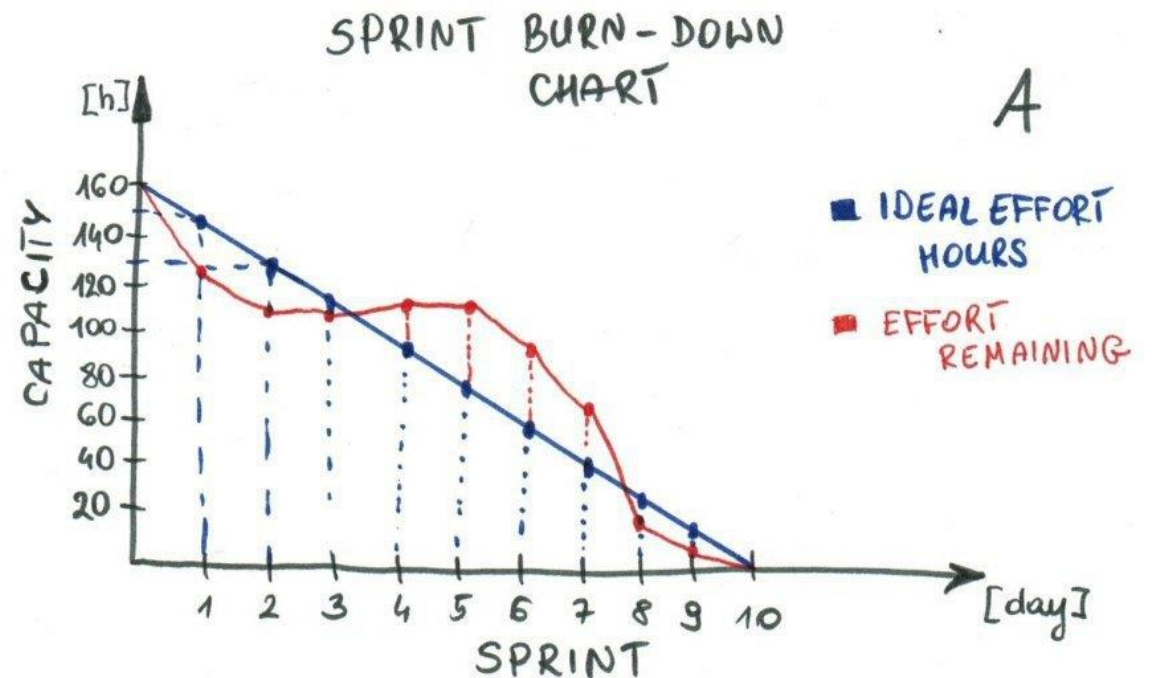
- El Sprint Burndown chart **hace visible el trabajo del equipo Scrum**: es una representación gráfica de la velocidad a la que se completa el trabajo y cuánto trabajo queda por hacer.
- Los equipos de scrum organizan el desarrollo en sprints con un tiempo asignado. Al inicio del sprint, el equipo plantea un pronóstico de la cantidad de trabajo que puede completar durante el sprint.
 - **Un informe de evolución de sprints supervisa la finalización del trabajo a lo largo del sprint.**
 - El eje de abscisas representa el tiempo
 - El eje de ordenadas se refiere a la cantidad de trabajo o esfuerzo que queda por completar, medido en puntos de historia u horas.
 - Puede ser evaluado en los eventos de las daily standups para entender el estado de las tareas, por equipo y por persona.

Métricas Ágiles: Cómo usar métricas ágiles para optimizar la entrega

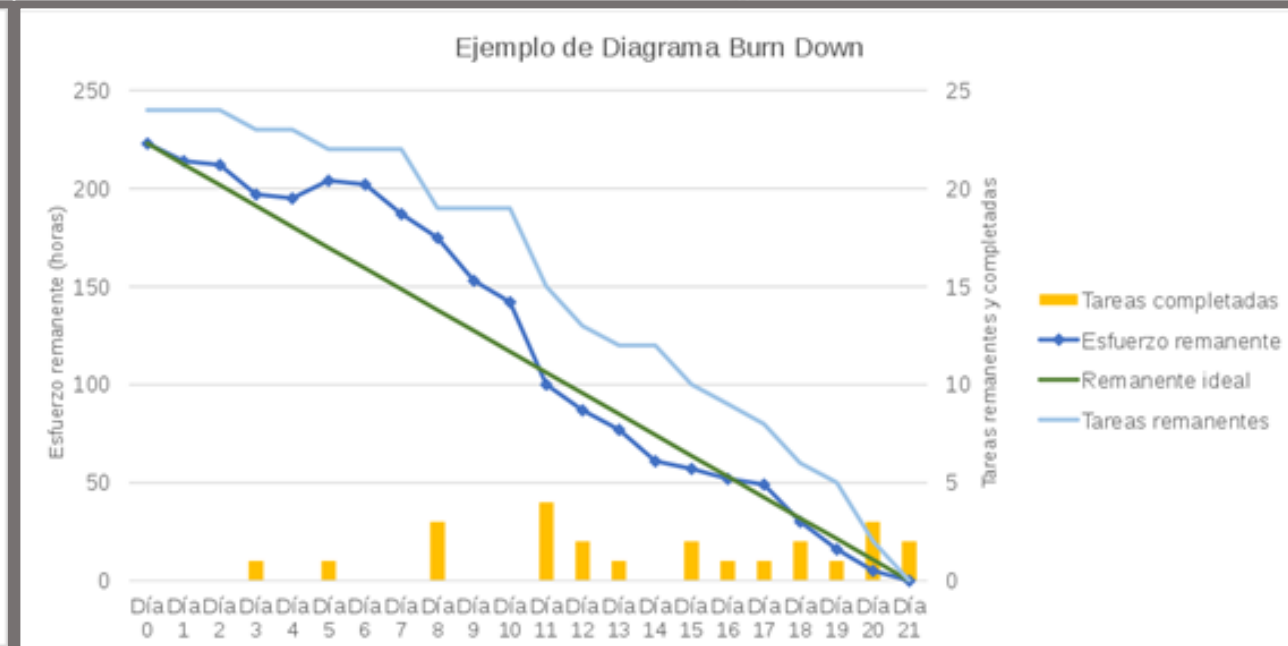
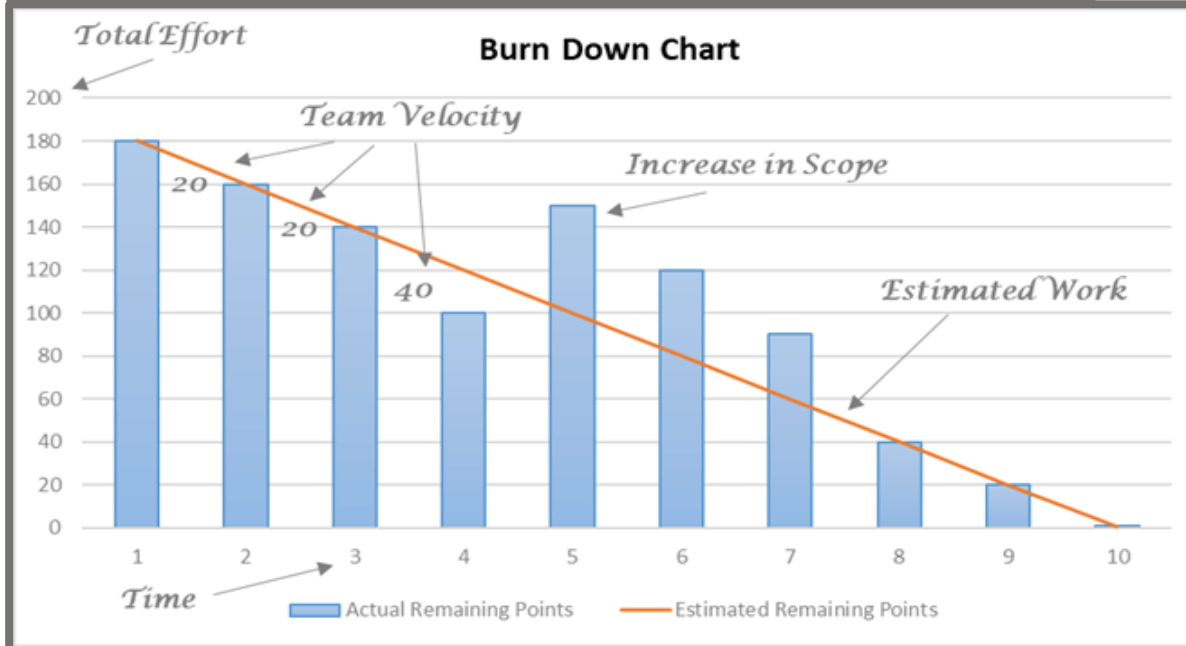
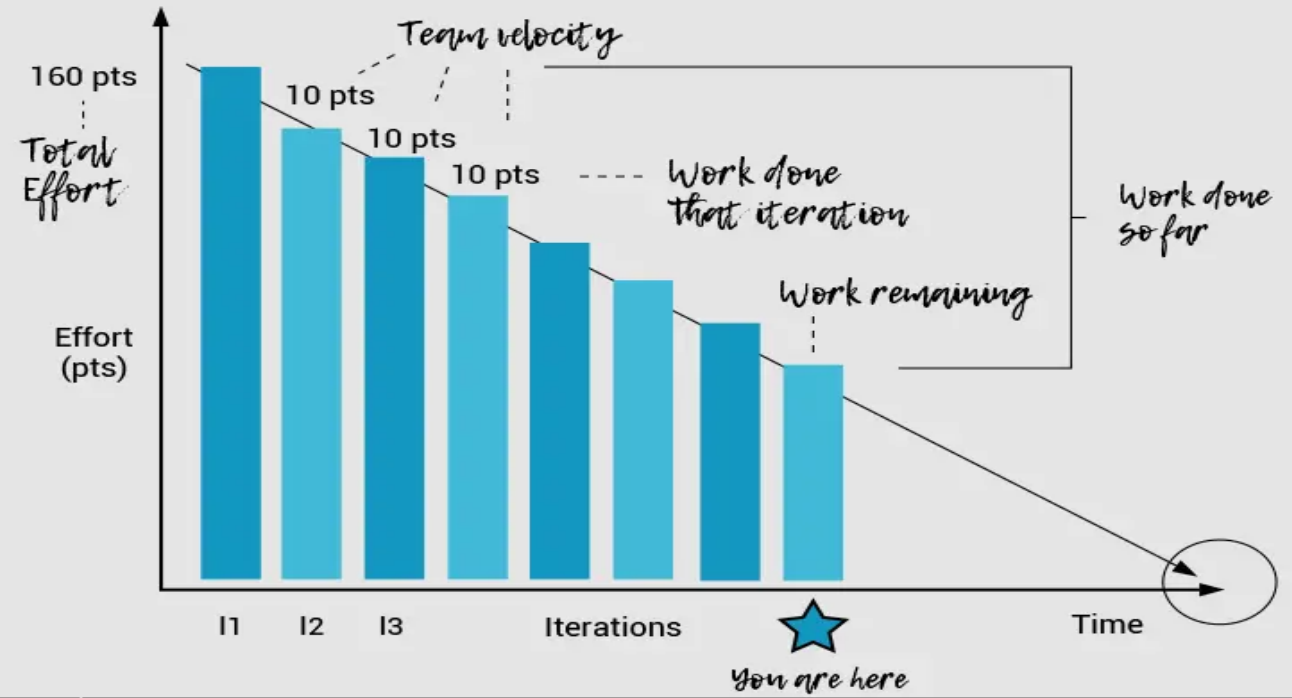
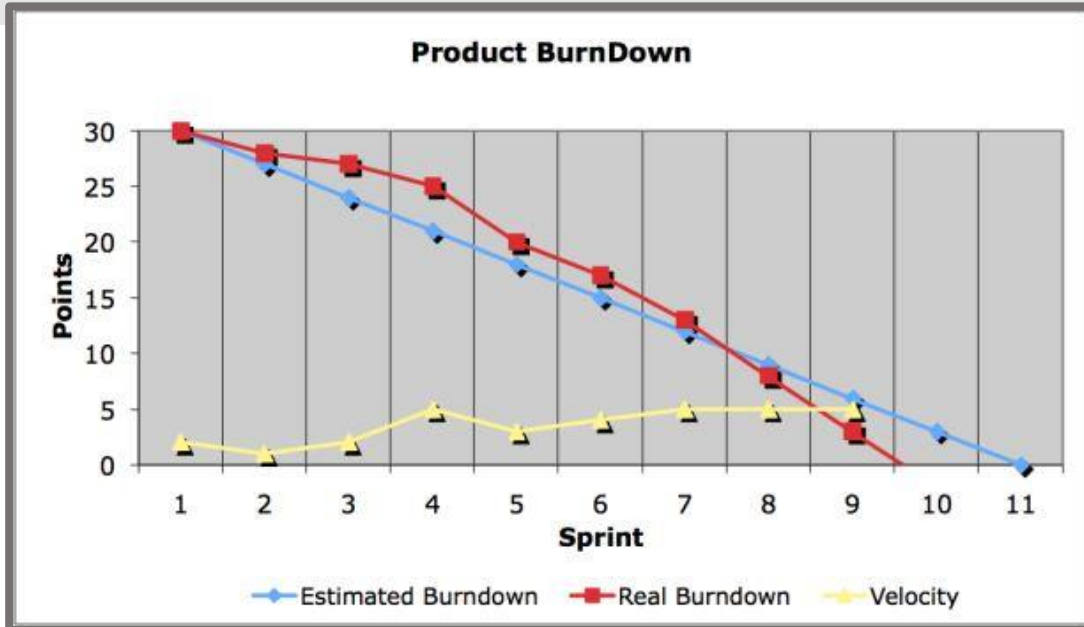
1- Sprint burndown (cont.)

Antipatrones ante los que estar alerta

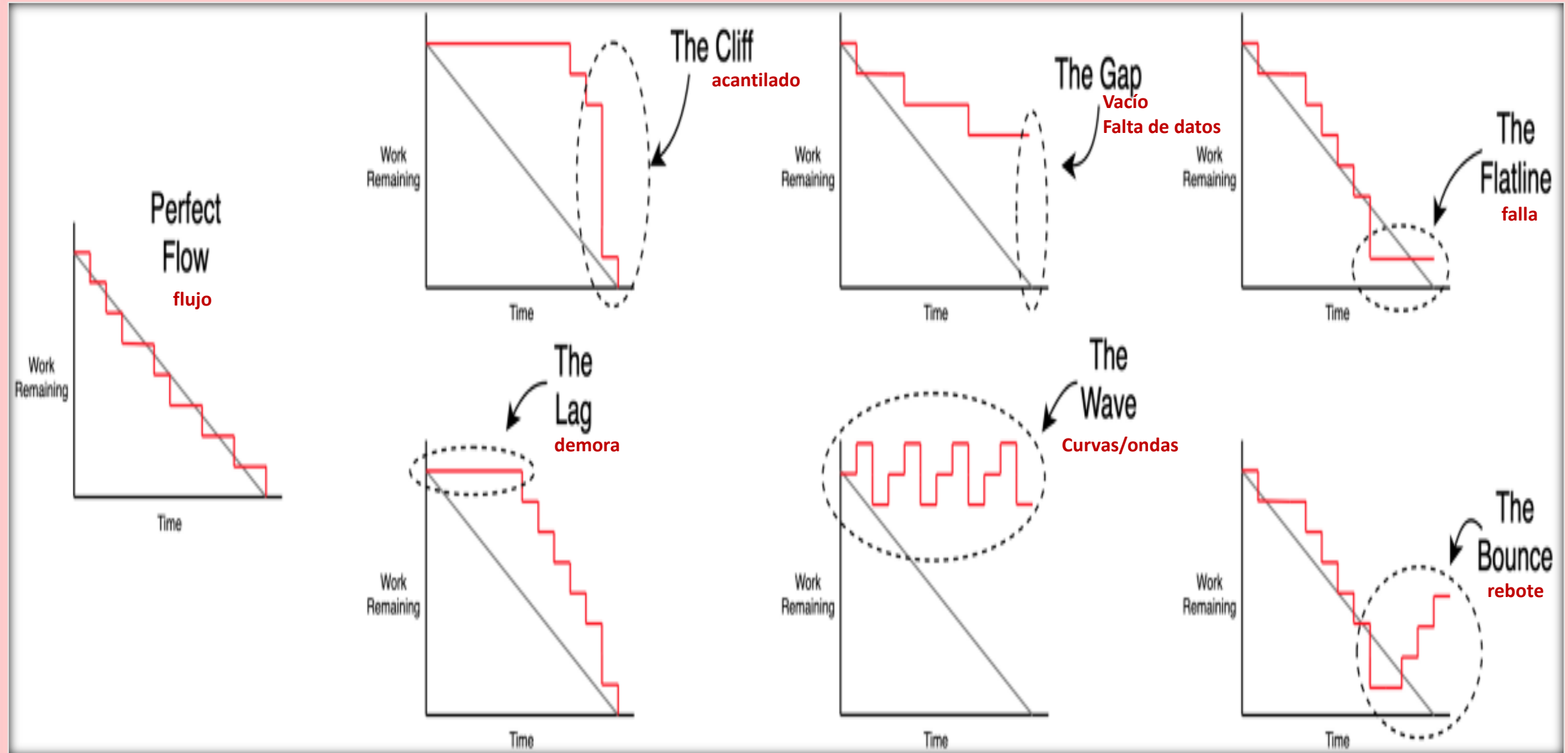
- * El equipo termina antes de tiempo todos los sprints porque **no están asumiendo trabajo suficiente**.
- * El equipo no cumple las previsiones sprint tras sprint porque **están asumiendo demasiado trabajo**.
- * La línea de evolución marca caídas pronunciadas en lugar de una evolución más gradual debido a que el trabajo no se ha dividido granularmente.
- * El propietario del producto añade o cambia el alcance en mitad del sprint.



Scrum Burndown chart



Scrum Burndown chart

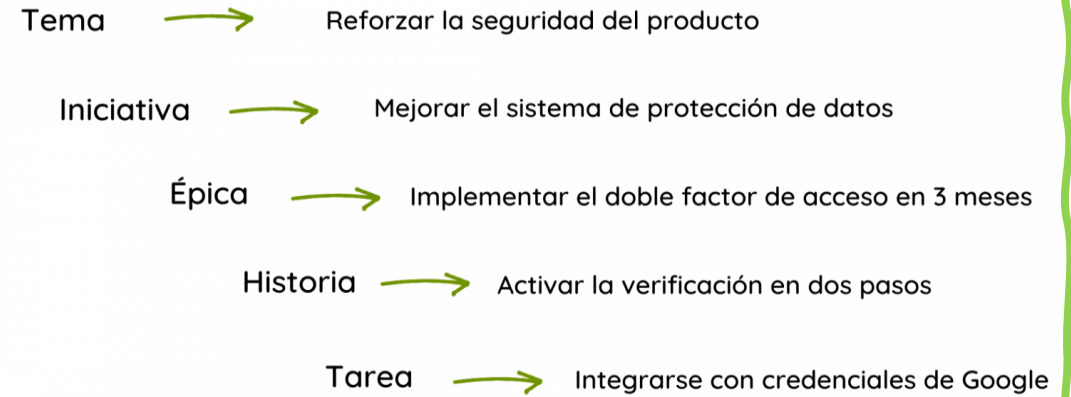


Desagregación en Scrum: repaso

Las épicas son un conjunto de historias de usuario, recogidas dentro de las iniciativas.

1. Se definen **Temas o Thems o Claims**, son estratégicos, generales, más a largo plazo y amplios.
2. Se definen **Iniciativa u Objetivos** comunes relacionados con un Tema, puede ser que una misma iniciativa comparta dos temas.
3. Definimos todas las **historias de usuario** que necesitamos para perseguir ese objetivo en común que hemos marcado en el Tema, armándose grupos de historias o **Épicas**. Este proceso lo hacemos en conjunto con los requerimientos de los clientes.
4. Dividimos **cada Epica en Historias de Usuario**. En este paso, no sólo la colaboración con el cliente es fundamental, sino que nos ponemos en su lugar para adaptar lo que queremos hacer a su lenguaje y tal y como él mismo nos lo pide. Por ejemplo, tener la posibilidad de vincular actividades entre sí, si estamos desarrollando un planificador online. 1 US: n Tareas (siendo $n \geq 1$).
5. Dividimos las Historias en **Tareas**
6. Luego se realizará la estimación de las tareas de cada US según sus tareas en la Planning

Ejemplo en un producto de software



Métricas Ágiles: Cómo usar métricas ágiles para optimizar la entrega

2- Evolución de Épicas y publicaciones

- Los diagramas de evolución de [épicas y versiones](#) sirven para realizar el seguimiento del progreso del desarrollo a lo largo de una muestra más amplia de trabajo que en la evolución de sprints, y guía el desarrollo de los equipos de scrum y kanban.
- Dado que un sprint (en los equipos de scrum) puede contener trabajo de varias épicas y versiones, **es importante supervisar el progreso de los sprints individuales, así como de las épicas y las versiones.**
- La **"corrupción del alcance" es la inyección de nuevos requisitos en un proyecto ya definido.**
 - Por ejemplo, si un equipo entrega un nuevo sitio web a la empresa, la corrupción del alcance pedirá nuevas funcionalidades después de que se haya realizado el esquema de los [requisitos](#) iniciales. Aunque tolerar la corrupción del alcance durante un sprint es mala práctica, los cambios en el alcance en las épicas y versiones son una consecuencia natural de un desarrollo ágil.
- A medida que el equipo avance por el proyecto, el PO puede decidir asumir o retirar trabajo en función del aprendizaje o cambios en el negocio, negociando con el equipo si hay cambios. Los diagramas de evolución de épicas y versiones informan a todo el mundo del **flujo de trabajo** dentro de la épica y la versión.

2- Evolución de epicas y publicaciones (cont.)

- **Antipatrones ante los que estar alerta**
 - * Las previsiones de epicas o versiones no se actualizan a medida que el equipo avanza por el trabajo.
 - * No se observa progreso después de varias iteraciones.
 - * Cuando el PO no entiende completamente el problema a solucionar.
 - * El alcance aumenta con mayor rapidez que la capacidad de atenderlo el equipo.
 - * El equipo no está lanzando versiones incrementales a lo largo del desarrollo de las epicas.

Diagrama de Evolución de Epicas y publicaciones



- 1 **Menú de Epicas:** selecciona el Epic del que quieras ver datos (en el ejemplo el #68)
- 2 **Trabajo añadido:** el segmento azul oscuro muestra la cantidad de trabajo añadido al Epic en cada sprint. En este ejemplo, el trabajo se mide en puntos de historia.
- 3 **Trabajo restante:** el segmento azul claro muestra la cantidad de trabajo restante en el Epic.
- 4 **Trabajo completado:** el segmento verde claro, representa la cantidad de trabajo que se ha completado para el Epic en cada sprint.
- 5 **Finalización planificada:** el informe planifica cuántos sprints se tardará en completar el Epic, según la velocidad del equipo.

Métricas Ágiles: Cómo usar métricas ágiles para optimizar la entrega

3- Velocidad

La velocidad es la cantidad media de trabajo que un equipo de scrum lleva a cabo durante un sprint, medida en puntos de historia u horas, y es muy útil para los pronósticos.

- El PO puede usar la velocidad para predecir la rapidez con la que un equipo puede recorrer el backlog, debido a que el informe supervisa el trabajo previsto y el completado a lo largo de varias iteraciones. **Cuantas más iteraciones se contemplen, más precisa será la previsión.**
 - Digamos que el PO desea completar 500 puntos de historia del backlog. Sabemos que el equipo de desarrollo generalmente completa 50 puntos de historia en cada iteración.
 - Es razonable que el PO suponga que el equipo necesitará 10 iteraciones (más o menos) para finalizar el trabajo requerido.
- Es importante supervisar la evolución de la velocidad a lo largo del tiempo. Los nuevos equipos seguramente vean un aumento en velocidad a medida que el equipo optimice las relaciones y los procesos de trabajo. Los equipos existentes pueden supervisar su velocidad para garantizar un rendimiento coherente a lo largo del tiempo, y pueden confirmar si un cambio en un proceso concreto ha mejorado algo o no. **Un descenso de la velocidad media generalmente es un indicador de que alguna parte del desarrollo del equipo se ha vuelto ineficiente y debe tratarse en la siguiente retrospectiva.**

3- Velocidad (cont.)

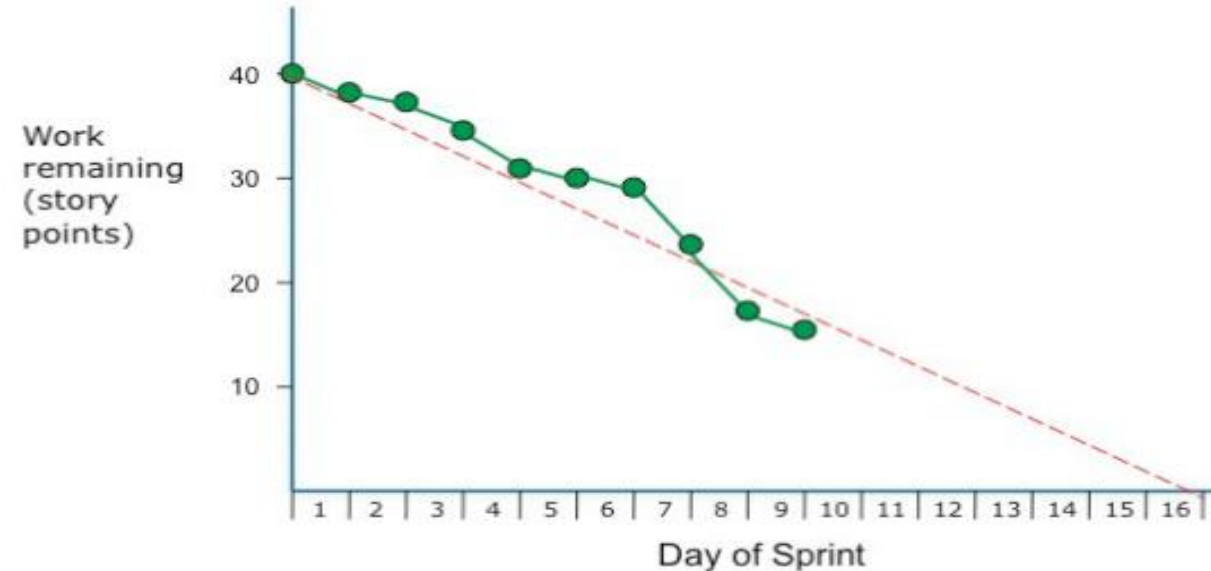
Antipatrones ante los que estar alerta

- ★ Si la velocidad experimenta muchas variaciones en un largo periodo de tiempo, revisa las prácticas de estimación del equipo.

En el evento de la retrospectiva del equipo, pregunta lo siguiente:

- Hay dificultades de desarrollo que no tuvimos en cuenta al estimar el trabajo? ¿Cómo podemos dividir mejor el trabajo para detectar algunas de estas dificultades?
- Hay presión empresarial externa que empuja al equipo más allá de sus límites? ¿Se están sacrificando las buenas prácticas de desarrollo debido a ello?
- Como equipo, ¿somos demasiado optimistas al realizar previsiones de los sprints?

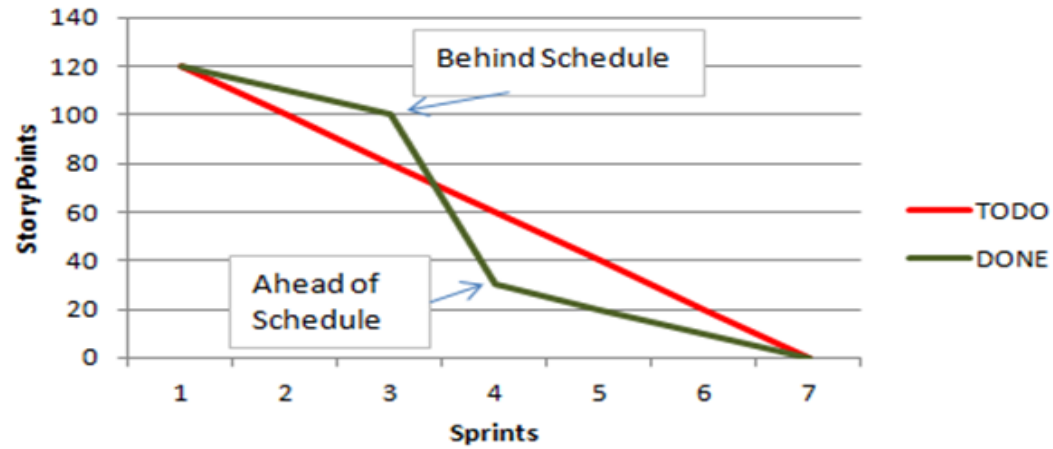
Velocity is Plotted on the "Burndown Chart"



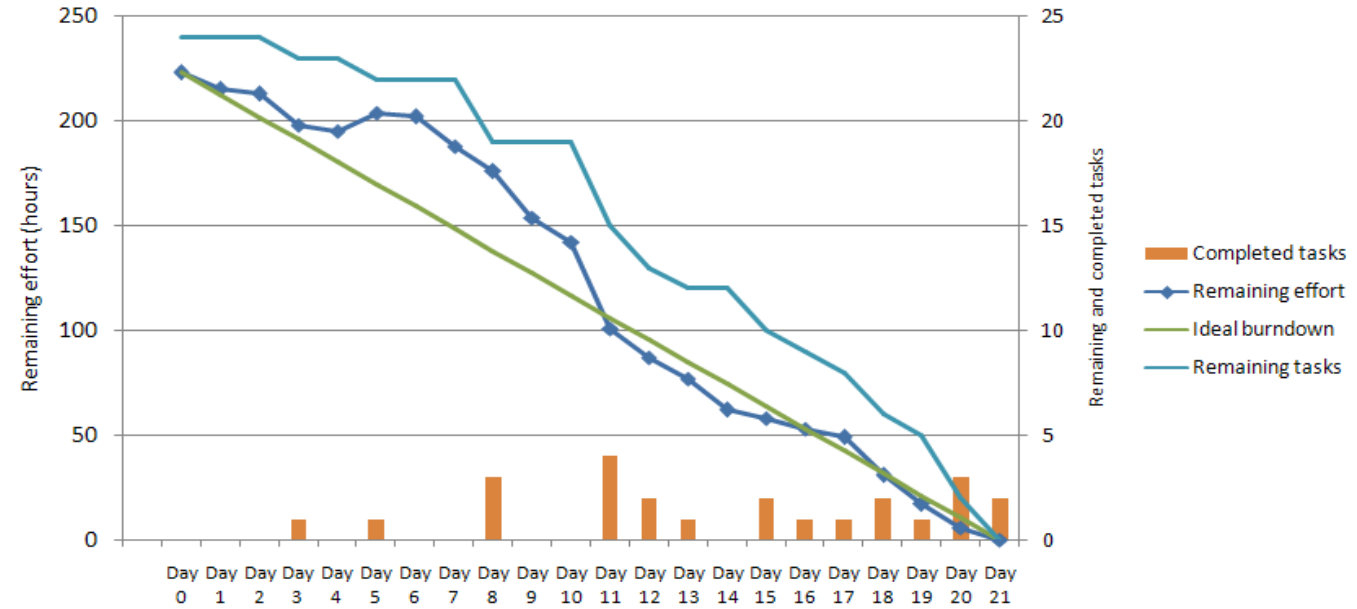
Burndown chart answers the question, "Are we on track to successfully deliver this sprint's output?"

Velocidad

Release Burndown Chart

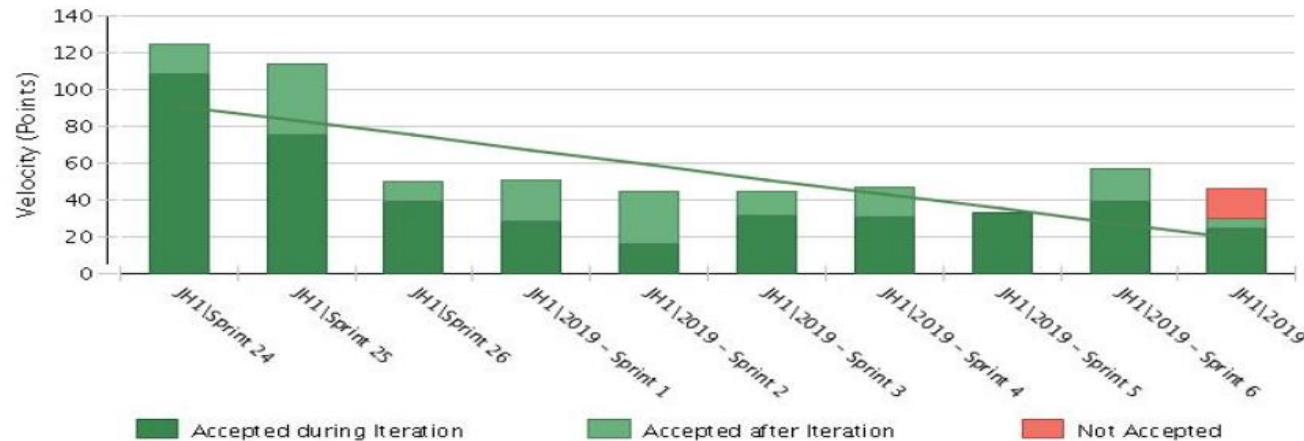
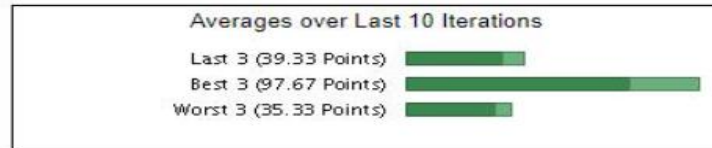


Sample Burndown Chart



Velocity Chart

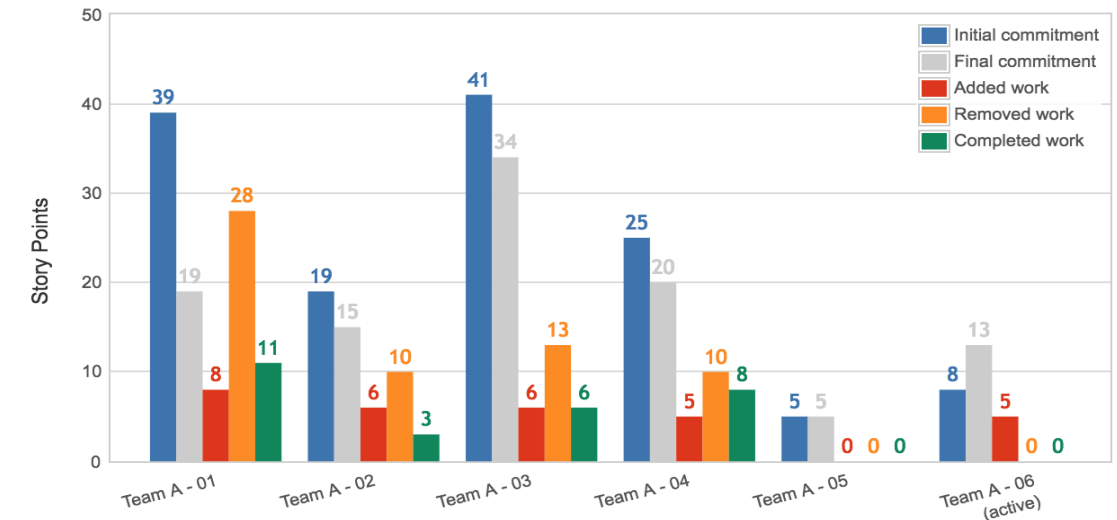
Team Lightning



Agile Velocity Chart

Team A - Velocity Chart

Export Support



Métricas Ágiles: Cómo usar métricas ágiles para optimizar la entrega

4-Gráfico de control

Los gráficos de control se centran en el tiempo de ciclo de los problemas individuales: el tiempo total desde "en progreso" hasta "terminado".

Es probable que los equipos con tiempos de ciclo más cortos tengan un mayor rendimiento, y los equipos con tiempos de ciclo consistentes en muchos problemas son más predecibles en la entrega del trabajo.

- Si bien el tiempo de ciclo es una métrica principal para los equipos Kanban, los equipos de Scrum también pueden beneficiarse de un tiempo de ciclo optimizado.
- Medir la duración del ciclo es un modo flexible y eficiente de mejorar los procesos de un equipo porque los resultados de los cambios se detectan casi instantáneamente, de modo que se permiten ajustes inmediatos.
- **El objetivo final es tener una duración del ciclo corta y homogénea, independientemente del tipo de trabajo** (nueva funcionalidad, deuda técnica, etc.).

4-Gráfico de control – cont.

- **Antipatrones ante los que estar alerta**
 - Los gráficos de control pueden parecer demasiado variables al principio. No te preocupes demasiado con cada dato que se salga de la norma. Busca tendencias. Estas son dos áreas a las que estar atentos:
 - ★ **Aumento de la duración del ciclo**: El aumento de la duración del ciclo mina la agilidad lograda con tanto esfuerzo por parte del equipo. En la retrospectiva del equipo, dedica tiempo a entender el motivo de este aumento. Una excepción: Si la definición que hace el equipo de "finalizado" ha aumentado, la duración del ciclo probablemente aumente también.
 - ★ **Duración del ciclo heterogénea**: El objetivo es tener una duración del ciclo homogénea de los elementos de trabajo que tengan valores de punto de historia similares. Filtra el gráfico de control para cada valor de punto de historia en busca de homogeneidad. Si la duración del ciclo es heterogénea en valores de punto de historia grandes y pequeños, dedica tiempo en la retrospectiva a examinar los aspectos que se han pasado por alto y a mejorar las estimaciones futuras.

Métricas Ágiles: Cómo usar métricas ágiles para optimizar la entrega

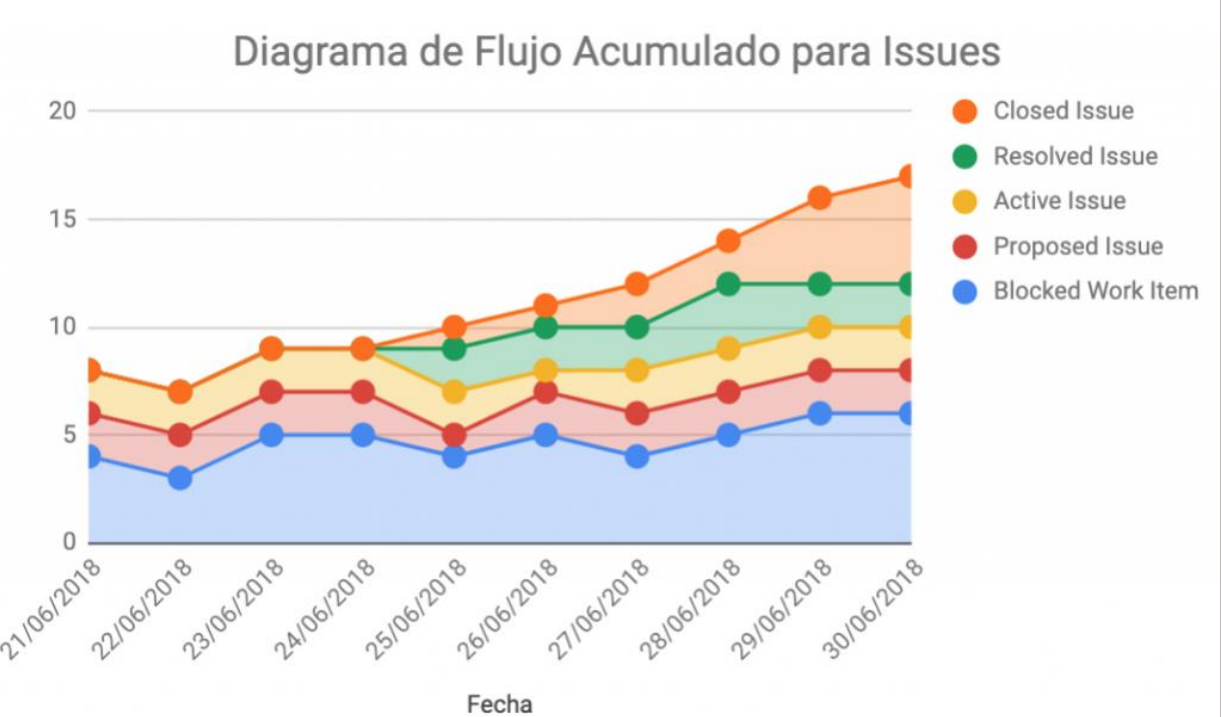
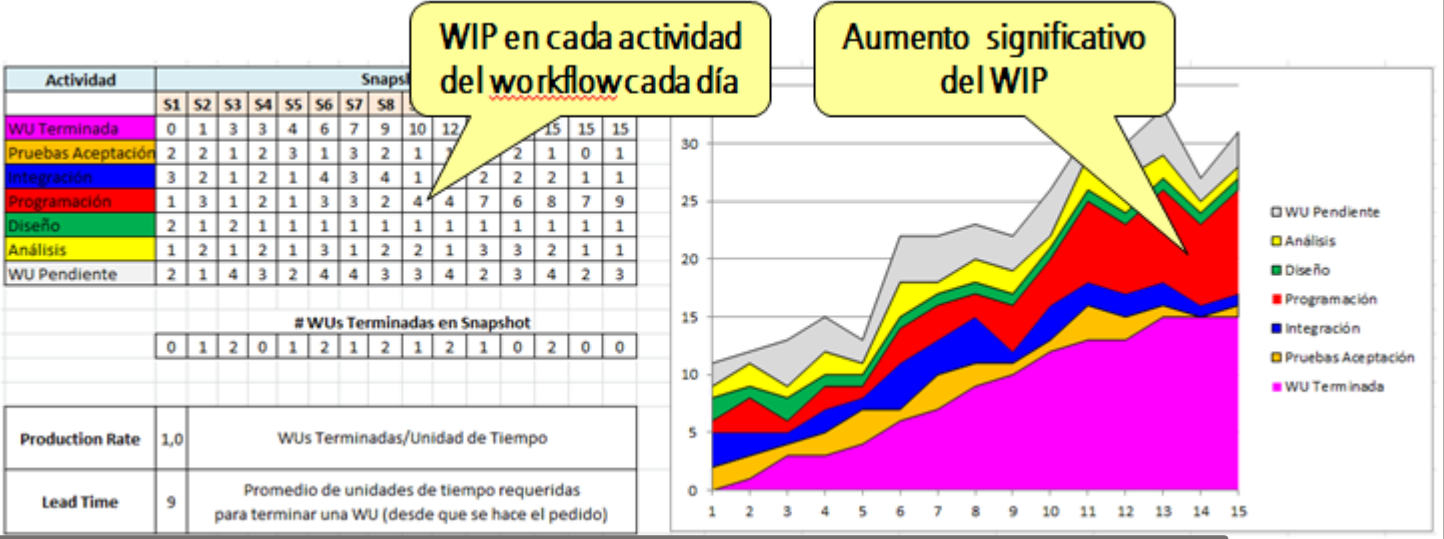
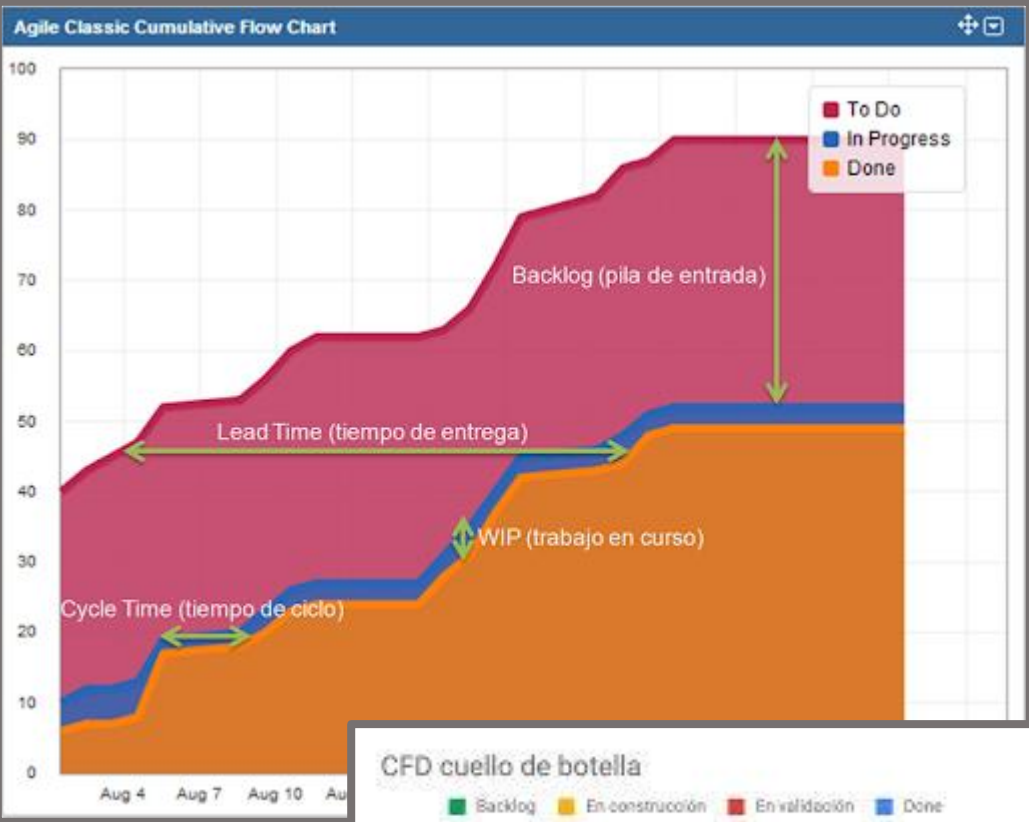
5-Diagrama de flujo acumulado

- El diagrama de flujo acumulado es un recurso clave para los equipos de [kanban](#). Ayuda a garantizar la coherencia del ritmo de trabajo en todo el equipo. Con el número de tickets en el eje de ordenadas, el tiempo en el eje de abscisas y colores para indicar los distintos estados del [workflow](#), indica visualmente las limitaciones y los cuellos de botella conjuntamente con los [límites del trabajo en curso](#).
- El diagrama de flujo acumulado debería ser una curva suave de izquierda a derecha. Las burbujas o huecos en cualquier color indican limitaciones y cuellos de botella.

Antipatrones ante los que estar alerta

- ✳ Las incidencias que causan bloqueos crean enormes copias de seguridad en algunas partes del proceso y muy escasas en otros.
- ✳ Crecimiento incontrolado del backlog a lo largo del tiempo. Esto da como resultado que los propietarios del producto no cierren incidencias obsoletas o que las incidencias con prioridad más baja no se traten nunca.

Diagrama de flujo acumulado



Métricas Agiles: más métricas

- Las buenas métricas no están limitadas a los informes mencionados anteriormente. Por ejemplo, la calidad es una métrica importante para los equipos ágiles y **hay varias métricas tradicionales que pueden aplicarse al desarrollo ágil**:
 - **Cuántos defectos se encuentran...**
 - durante el desarrollo?
 - después de la publicación al cliente?
 - por personas ajenas al equipo?
 - **Cuántos defectos se aplazan para una versión futura?**
 - **Cuántas solicitudes de servicio al cliente están llegando?**
 - **Cuál es el porcentaje de cobertura de las pruebas automatizadas?**
- Los equipos ágiles también deben tener en cuenta la frecuencia de las publicaciones y la velocidad de entrega. Al final de cada sprint, **el equipo debe publicar software a producción**:
 - Con qué frecuencia ocurre eso? Se están lanzando la mayoría de las compilaciones de versiones? Cuánto tarda el equipo en publicar una corrección urgente a producción? La publicación es fácil para el equipo o requiere actuaciones heroicas?
- **Las métricas son solo una parte para crear la cultura de equipo.**
 - Aportan un análisis cuantitativo del rendimiento del equipo y proporcionan objetivos mensurables. Aunque son importantes, tampoco te obsesiones con ellas. Escuchar el feedback durante las [retrospectivas](#) es igual de importante para aumentar la confianza del equipo, la calidad del producto y la velocidad de desarrollo en todo el proceso de publicación. Se debe usar el feedback cuantitativo y cualitativo para impulsar los cambios.



Métricas Agiles

Velocidad, trabajo y tiempo son las tres magnitudes que mide la gestión de proyectos ágil, y componen la fórmula de la velocidad, definiéndola como: cantidad de trabajo realizada en por unidad de tiempo.

• **Velocidad = Trabajo / Tiempo**

Tiempo

- El desarrollo ágil emplea la técnica “timeboxing” para gestión de tiempo. En el caso de Scrum, la unidad de tiempo para cada incremento de producto es el Sprint.

Tiempo real y tiempo ideal

- Antes de continuar, una observación: la diferencia, al hablar de tiempo, **entre tiempo “real” y tiempo “ideal”**.
- **Tiempo real, es el efectivo de trabajo**. Es equivalente a la jornada laboral. Para un equipo de cuatro personas con jornada laboral de ocho horas el tiempo real en una semana (cinco días laborables) es: $4 * 8 * 5 = 160$ horas. El tiempo real de ese equipo en un sprint de 12 días de trabajo es: $4 * 8 * 12 = 384$ horas
- Sin embargo el término “**tiempo ideal**” se refiere al tiempo de trabajo necesario, en “condiciones ideales”, esto es, sin ninguna interrupción, pausa, distracción o atención a tareas ajenas a la tarea del sprint que se tiene asignada.



Métricas Agiles

Velocidad (Continuando con lo anterior)

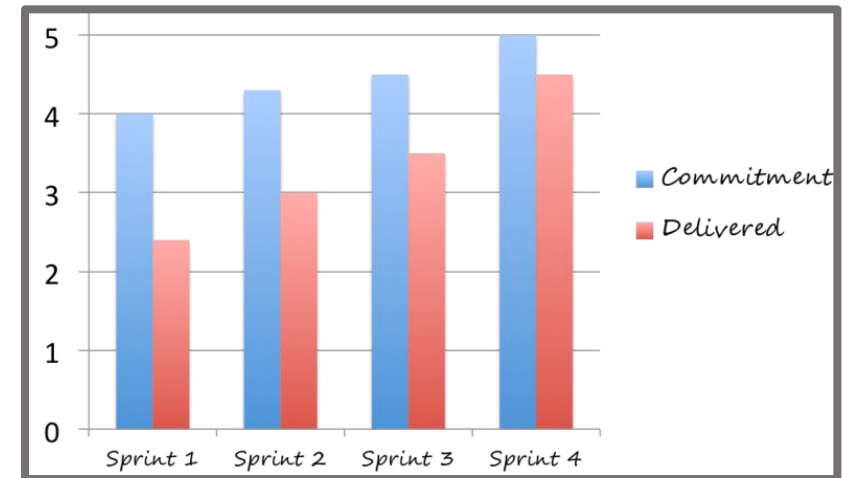
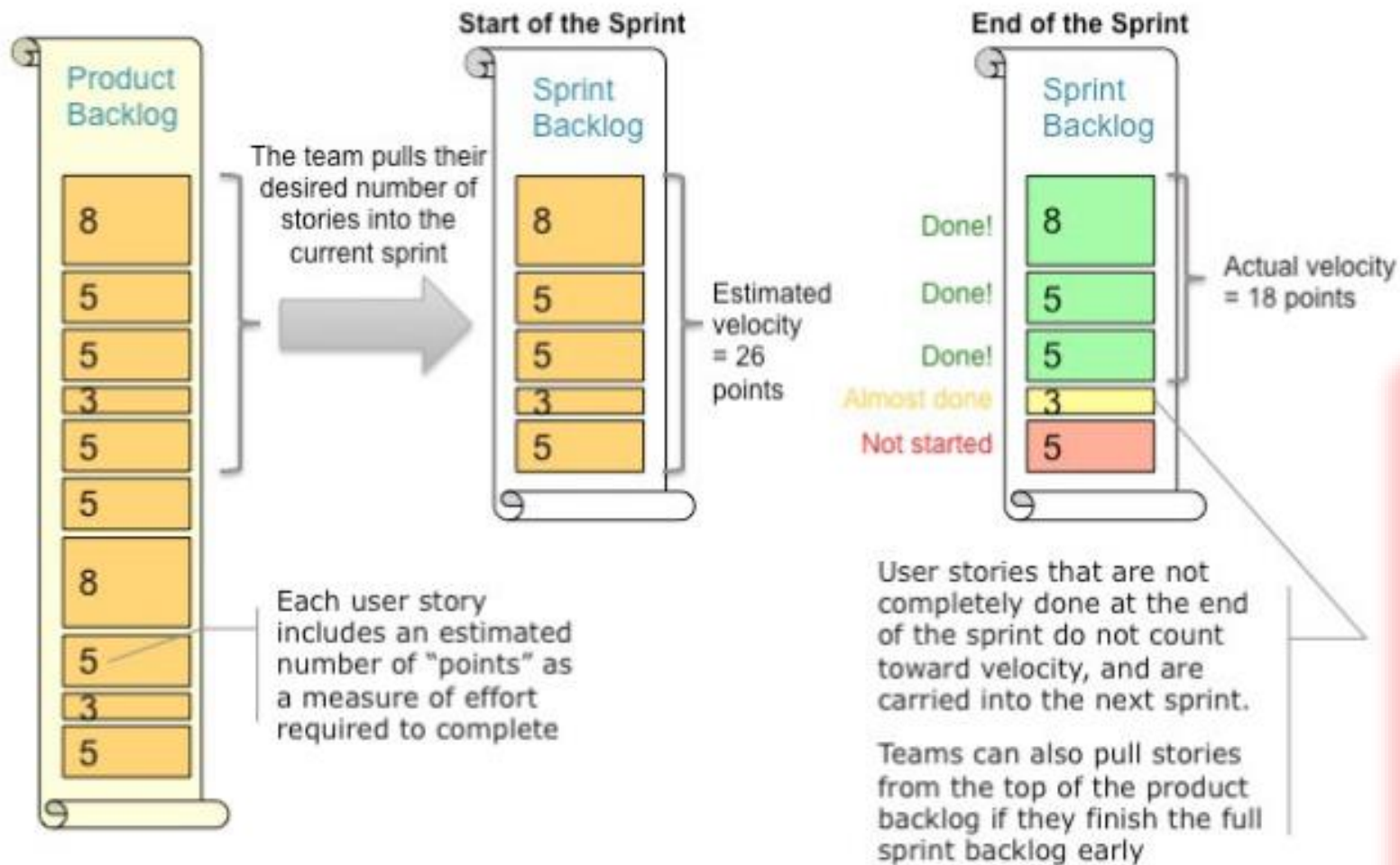
Velocidad es la magnitud que viene determinada por la cantidad de trabajo realizada en un periodo de tiempo (Timebox). Los equipos que miden el trabajo con tiempo ideal, hablan de “Velocidad”.

- Ejemplo: si decimos, la velocidad del equipo en el sprint ha sido de 23, se refiere a que han completado tareas que medían en total 23 story points.
- Si en el sprint siguiente consiguen una velocidad mayor, querrá decir que han **logrado programar más story points en el mismo tiempo**, o lo que es lo mismo que han conseguido aumentar el porcentaje de tiempo ideal en la jornada de trabajo: han reducido los tiempos de interrupciones, distracciones o dedicados a otras tareas.



Velocidad

"Velocity" is the Key Metric in Scrum



Burndown Strategies

- **Burndown Stories** solo en Puntos de historia de Usuario: deben ser historias muy chicas para aplicarlo
- **Burndown tasks en puntos:** usar planning póker para estimar las tareas en puntos
- **Burndown tasks en horas:** trata de **no aplicarse**, **no se ve como una Buena práctica**

Métricas Agiles

Trabajo

- Medir el trabajo puede ser necesario por dos razones: **para registrar el ya hecho, o para estimar anticipadamente, el que hay que realizar.** En ambos casos se necesita una unidad, y un criterio objetivo de cómo se cuantifica.

Sprint

$$\text{Velocidad} = \text{Trabajo} / \text{Tiempo}$$



Trabajo ya realizado

- Medir el trabajo ya realizado no entraña especial dificultad. Se puede hacer con unidades relativas al producto (p. ej. líneas de código) o a los recursos empleados (coste, tiempo de trabajo...)
- Para medirlo basta contabilizar lo ya realizado, empleando las unidades con las que se opere: líneas de código, horas trabajadas (reales o teóricas)...

Métricas Ágiles

Trabajo pendiente de realizar

- Scrum mide el trabajo pendiente para:
 - Estimar el esfuerzo y la duración prevista para cada tarea.
 - Determinar el avance del proyecto, y en especial de cada sprint.

$$\text{Velocidad} = \text{Trabajo} / \text{Tiempo}$$



- ***La estrategia empleada por la gestión ágil es:***

- No empeñarse en estimaciones precisas.
- Estimar con la técnica “juicio de expertos”
- Descomponer las tareas de los sprints en sub-tareas más pequeñas, si las estimaciones superan los rangos de las 16-20 horas de trabajo.

Unidades de trabajo

- Las unidades para medir el trabajo pueden estar relacionadas directamente con el producto, como los tradicionales puntos de función de COCOMO; o indirectamente, a través del tiempo necesario para realizarlo.

Métricas Ágiles

Unidades de trabajo (Cont.)

- La gestión ágil suele llamar a las unidades que emplea para medir el trabajo “puntos”, “puntos de funcionalidad” “puntos de historia”...
- Cada organización, según sus circunstancias y su criterio institucionaliza su métrica de trabajo definiendo el nombre y las unidades. Puede basarse en:
 - Estimación del tamaño relativo y emplear puntos
 - Estimación del tiempo ideal necesario para realizar la tarea que se mide.
- Lo importante es que la métrica empleada, su significado y la forma de aplicación sea consistente en todas las mediciones, en todos los proyectos de la organización y conocida por todas las personas: ***Que se trate de un procedimiento de trabajo institucionalizado.***



Velocidad

- Velocidad es la magnitud que viene determinada por la cantidad de trabajo realizada en un periodo de tiempo (Timebox) Los equipos que miden el trabajo con tiempo ideal, hablan de “Velocidad”.
 - Decir, por ejemplo, que la velocidad del equipo en el sprint ha sido de 23, se refiere a que han completado tareas que medían en total 23 story points.
 - Si en el sprint siguiente consiguen una velocidad mayor, querrá decir que han **logrado programar más story points en el mismo tiempo**, o lo que es lo mismo que han conseguido aumentar el porcentaje de tiempo ideal en la jornada de trabajo: han reducido los tiempos de interrupciones, distracciones o dedicados a otras tareas.



Métricas: conclusiones

La estrategia empleada por la gestión ágil es:

- No es posible estimar con precisión, ni el trabajo de un requisito, ni el tiempo necesario para desarrollarlo.
- La complejidad de las técnicas de estimación crece exponencialmente en la medida que:
 - Intentan incrementar la fiabilidad y precisión de los resultados.
 - Aumenta el tamaño de la tarea estimada.
- No profundiza en estimaciones precisas.
- Emplea la técnica de “juicio de expertos”
- Descompone las tareas de los sprints en sub-tareas más pequeñas, si las estimaciones superan los rangos de las 16-20 horas de de trabajo.
- **Velocidad y eficiencia son términos similares. Se suele preferir “velocidad” cuando las mediciones se basan en tiempo teóricos, y “eficiencia” cuando lo hacen en tiempo real.**



Métricas Ágiles

- **Tiempo real o tiempo de trabajo:** Tiempo efectivo para realizar un trabajo. Se suele medir en horas o días.
- **Tiempo teórico o tiempo de tarea:** Tiempo que sería necesario para realizar un trabajo en “condiciones ideales”: si no se produjera ninguna interrupción, llamadas telefónicas, descansos, reuniones, etc.
- **Puntos de función o puntos de funcionalidad:** Unidad de medida relativa para determinar la cantidad de trabajo necesaria para construir una funcionalidad o historia de usuario del product backlog.



Métricas Ágiles

- **Estimaciones:**
 - Cálculo del esfuerzo que se prevé necesario para desarrollar una funcionalidad.
 - Las estimaciones se pueden calcular en unidades relativas (puntos de función) o en unidades absolutas (tiempo teórico).
- **Velocidad absoluta:** cantidad de producto construido en un sprint. Se expresa en la misma unidad en la que se realizan las estimaciones (puntos de función, horas o días reales o teóricos).
- **Velocidad relativa:**
 - Cantidad de producto construido en una unidad de tiempo de trabajo.
 - P. ej.: puntos de función / semana de trabajo real; o horas teóricas / día de trabajo real...

